IITK INNOVATION CHALLENGE 2026
Qualification Report

yuri daisuki - IIT Roorkee

Taufeeque Sifat
taufeeque_as@ce.iitr.ac.in
IIT Roorkee

Shivam
shivam1@me.iitr.ac.in
IIT Roorkee

Divyam Kumar
divyam_k@ee.iitr.ac.in
IIT Roorkee

Abstract: This report analyzes power traces from an AES-128 implementation to recover a countermeasure-induced fake round-10 key, estimate the trace count required for its recovery, infer the actual round-10 key, and derive the AES-128 master key. For fake key recovery, correlation power analysis (CPA) is applied using a transition Hamming-distance leakage hypothesis over ciphertext bytes and trace samples. trace count performance is measured by ranking each reference key byte as the number of traces varies. The first tested trace count at which all 16 reference bytes reach rank 1 is 1,933; rank 1 is then maintained for 100 consecutive tested counts beginning at 2,020. To infer the actual round-10 key, the analysis identifies absent ciphertext byte values, detects round activity from across-trace variance, and ranks shared missing S-box input candidates using transition-leakage correlations for rounds 1–8. The selected candidate is $z = \texttt{0xFF}$, yielding the round-10 key `d014f9a8c9ee2589e13f0cc8b6630ca6`. Reversing the AES-128 key schedule gives the master-key candidate `2b7e151628aed2a6abf7158809cf4f3c`. The actual round-key inference depends on the shared-endpoint and leakage-model assumptions used in the candidate scan.

I. Introduction

Power consumption can vary with the data a processor reads, writes, and transforms. A power trace therefore contains information about internal computation, even when the cryptographic output alone does not reveal it. Side-channel analysis uses a model of that computation to compare predicted intermediate values with measurements. Correlation power analysis (CPA) is one way to rank key guesses by how closely their predicted leakage follows the recorded traces [1], [2].

This report studies a trace challenge, with the premise being an AES-128 implementation on an embedded device and its power traces for the 10th round provided to us, along with four related tasks. There is a faking countermeasure as well, which aims to protect the implementation against conventional CPA, with one such countermeasure defined in [4].

The first asks for a fake round-10 key from the final-round leakage. The second asks how the number of traces affects recovery of that key and to report the minimum number of traces for effective recovery fo the fake key. The third asks for the actual round-10 key, despite the fake key behavior at the final round. The fourth asks for the AES-128 master key. The analysis uses CPA for the first two tasks, then combines ciphertext-frequency observations with leakage from earlier AES rounds for the third. The final task reverses the AES-128 key schedule [6].

Side-channel based reverse engineering uses physical leakage to infer internal algorithm or implementation details. Novak studies this use of side channels through sign-extended differential power analysis and attacks on table lookups [5]. Task 3 fits within this broader category because we use ciphertext statistics and power traces to test hypotheses about the faking countermeasure. Here, we know that the device uses AES; the inference concerns the countermeasure's lookup-table behavior and the actual round key.

The fake key attack uses a transition Hamming-distance hypothesis and searches all 200 samples for each key-byte guess. For the trace count study, the byte ranks are recomputed on prefixes of one seeded shuffle of the capture. For the actual round-10 key, the analysis first finds ciphertext byte values that are absent from the capture, then tests the corresponding endpoint candidates against transition leakage at detected round-activity samples. The selected round key is reversed through the AES-128 key schedule to obtain a master-key candidate.

The results support the fake round-10 key `91dec068e3face4599ce38f420863a30`. In the saved trace count characterization, all 16 bytes first reach rank 1 at 1,933 traces; they then remain at rank 1 for 100 successive tested counts beginning at 2,020. The selected actual round-10 key is `d014f9a8c9ee2589e13f0cc8b6630ca6`, and its reversed master-key candidate is `2b7e151628aed2a6abf7158809cf4f3c`. The actual-key result is evidence under the leakage and shared-endpoint assumptions described below, rather than a proof independent of those assumptions.

Section II reviews the AES and leakage-model details needed for the analysis. Section III describes the trace data and experimental settings. Section IV gives the method used for each task, and Section V presents the results. Section VI discusses limitations, followed by the conclusion in Section VII.

Figure 1 gives the variance profile used to locate round activity in the traces.

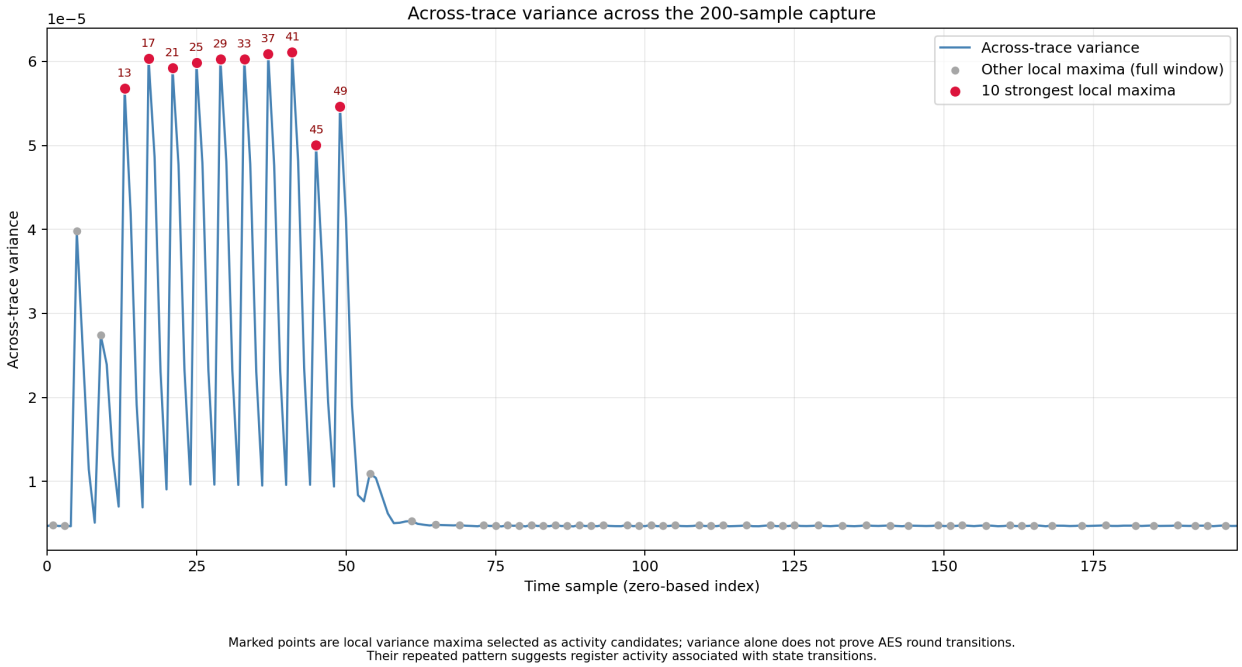


Fig. 1. Across-trace variance over the 200-sample capture. The selected local maxima indicate candidate activity times.

II. Background and Problem Formulation

A. AES-128 and the Round-10 Key

AES-128 encrypts a 128-bit block using an initial key addition followed by ten rounds. The AES-128 key schedule expands the 128-bit master key into eleven 128-bit round keys. The final ciphertext is produced by the tenth round and its round key, commonly denoted RK_{10} [6]. Because the key schedule is reversible, a candidate RK_{10} determines a candidate master key.

The challenge's final-round leakage is associated with a fake key. Consequently, applying a final-round CPA model can recover the fake key without directly revealing the actual RK_{10}. Tasks 3 and 4 use ciphertext statistics and leakage associated with earlier rounds to pursue the actual round key and then the master key.

B. Leakage Model and CPA

The analysis assumes first-order leakage related to the Hamming distance (HD) between two byte values. For a byte value x, its Hamming weight, $HW(x)$, is the number of set bits. A transition from x to y is modeled by $HW(x \oplus y)$, where $\oplus$ denotes XOR. This is a simplified leakage model. Real measurements can also contain noise, unrelated activity, and effects not captured by the model.

For Task 1, let $C_{j,b}$ be ciphertext byte b for trace j, and let $\pi(b)$ identify the ciphertext byte routed to the corresponding pre-output state position by the AES ShiftRows mapping. For key guess k, the predicted final-round transition is

$$H_{j,b}(k) = HW\big(\mathrm{InvSBox}(C_{j,b} \oplus k) \oplus C_{j,\pi(b)}\big)\,. \tag{1}$$

For a hypothesis vector $H_b(k)$ and the measured vector $P(t)$ at sample t, the Pearson correlation is [3]

$$\rho\big(H_b(k), P(t)\big) = \frac{\sum_{j=1}^{n} \big(H_{j,b}(k) - \overline{H}_{b,k}\big)\big(P_j(t) - \overline{P}_t\big)}{\sqrt{\sum_{j=1}^{n} \big(H_{j,b}(k) - \overline{H}_{b,k}\big)^2}\sqrt{\sum_{j=1}^{n} \big(P_j(t) - \overline{P}_t\big)^2}}, \tag{2}$$

where n is the number of traces, $\overline{H}_{b,k}$ is the mean predicted leakage for guess k, and $\overline{P}_t$ is the mean measured value at sample t. The correlation is the centered covariance divided by the product of the two centered vector norms. The guess score is the largest absolute value of Pearson correlation over the 200 trace samples:

$$S_b(k) = \max_{t \in \{0,\ldots,199\}} \big|\rho\big(H_b(k), P(t)\big)\big|\,, \tag{3}$$

where $P(t)$ is the vector of measured values at sample t across traces. The most-correct byte guess maximizes $S_b(k)$.

C. Recovering the actual master key

Since AES-128 key round schedule is reversible, getting the round-10 key will help in getting the initial AES master key. Let assume that expanded key is a 44 word , 4 bytes for each word, $w[0], w[1], \ldots, w[43]$ [7]. The first four words form the master key, while the last four form the round-10 key. The subsequent keys, after the master key, are calculated using a recurrence relation,

$$w[i] = w[i-4] \oplus F_i(w[i-1]), \tag{4}$$

At every fourth word, F_i applies byte rotation, the forward S-box, and the round constant while at other positions, it leaves its input unchanged. Mathematically:

$$F_i(x) = \begin{cases} \mathrm{SubWord}(\mathrm{RotWord}(x)) & i \equiv 0 \pmod 4, \\ \quad \oplus\, Rcon[i/4], & \\ x, & \text{otherwise.} \end{cases} \tag{5}$$

Here, the rotation, RotWord, moves the first byte to the end of the four-byte word. The substitution, SubWord, replaces each byte with its value from the forward AES S-box. The word $Rcon[r]$ contains the constant for round r in its first byte and zeros in the remaining bytes [6].

XORing both sides with $F_i(w[i-1])$ and setting $j = i - 4$ gives backward rule to calculate previous round keys from the round-10 key [7]:

$$w[j] = w[j+4] \oplus F_{j+4}(w[j+3]). \tag{6}$$

Starting with $w[40], \ldots, w[43]$, we calculate $w[39]$ and continue backward to $w[0]$. Each step uses words that we already know. The resulting $w[0], \ldots, w[3]$ give the master key corresponding to the actual round-10 key.

III. Dataset and Experimental Setup

The capture in `Round10_traces.csv` contains 100,000 records. Each record has a 16-byte ciphertext followed by 200 trace samples. Task 1 and Task 2 evaluate the CPA hypothesis over all 200 samples. Task 3 reads the capture directly, detects activity samples from across-trace variance, and excludes a row if any ciphertext byte equals the doubled value identified at that byte position. The remaining rows are used to reconstruct candidate AES states.

Task 2 uses NumPy's pseudorandom generator with seed `0x5CA` to permute the trace rows once. The fixed seed the shuffled ordering so that repeated runs use the same subsets and reproduce the experiment. This seed has no cryptographic value and does not affect the retrieval of the key, although different orderings can change the trace count required for recovery. It tests increasing prefixes, beginning at 250 traces, and records the rank of each byte of the full-data Task 1 key. The tested trace count stops after the worst byte rank has remained one for 100 consecutive counts, showing that the worst rank increases even after it has achieved the minimum value.

The analysis scripts are in `task_solution/`: `first_task.py`, `second_task.py`, `third_task.py`, and `fourth_task.py`. Task 2 saves its rank plots in `rank_plots/rank_plot_all_bytes.png` and `worst_rank_plots/worst_rank_plot.png`, and its table in `trace_characterisation.csv`. Task 3 writes byte-wise candidate correlation plots to `actual_round10_correlation_plots/`. The variance plotting script, `task_solution/variance_plot.py`, saves its output in `variance_plots/variance_plot.png`.

IV. Methodology

A. Task 1: Fake Round-10 Key Recovery

For each of the 16 byte positions, the analysis evaluates all 256 byte key guesses. Equation (1) gives the predicted transition leakage for a guess while Equation (3) gives its score across the sample window. The key byte with the largest score is selected independently at each position. Concatenating the 16 winning bytes gives the fake round-10 key.

B. Task 2: trace count Characterization

The reference key is the 16-byte result of the Task 1 attack on the full-data. For each tested trace count N, the analysis uses the first N rows from the fixed shuffled ordering and recomputes the CPA score for each byte guess using all 200 samples. For reference byte F_b, its rank is

$$r_b(N) = 1 + \left|\{k : S_{b,N}(k) > S_{b,N}(F_b)\}\right|, \tag{7}$$

where $S_{b,N}(k)$ is the CPA score for guess k using the first N shuffled traces. The worst rank is $R(N) = \max_{0 \le b < 16} r_b(N)$, so all bytes rank first exactly when $R(N) = 1$.

The script reports two useful counts. The first full-recovery count is $\min\{N : R(N) = 1\}$ over tested counts. The stable count is the first tested N for which $R(N), R(N+1), \ldots, R(N+99)$ are all one. These values need not match because rankings can change as more traces are added.

C. Task 3: Actual Round-10 Key Recovery

For each ciphertext byte position, a histogram identifies an absent value M_b and a most frequent (doubled) value D_b. Let F_b be the fake key byte recovered in Task 1. Across all byte positions, the observed values satisfy

$$D_b \oplus F_b = SBox(0) = \texttt{0x63}. \tag{8}$$

This repeated relation connects the doubled ciphertext output to the Task 1 fake key and is consistent with a repeated lookup-table output associated with input zero. It describes the doubled values but does not identify the absent input or determine the actual round key by itself.

Trace timing is estimated separately from the key guesses. For sample t, across-trace variance is

$$V(t) = \frac{1}{N} \sum_{j=1}^{N} \left(P_j(t) - \overline{P}(t)\right)^2, \qquad \overline{P}(t) = \frac{1}{N} \sum_{j=1}^{N} P_j(t). \tag{9}$$

The script finds local maxima of $V(t)$, selects the ten strongest, and places them in time order. In this capture they occur at samples 13, 17, 21, 25, 29, 33, 37, 41, 45, and 49. The four-sample spacing is consistent with repeated state-update activity. We use the first eight selected variance peaks to score the predicted state transitions for AES rounds 1-8. This assumes that the peaks correspond to successive rounds, which variance alone cannot confirm.

The shared-endpoint assumption is that the same missing S-box input z explains the absent ciphertext output at all byte positions. Since the final AES round maps the relevant input

through the S-box and XORs the round key, each candidate z determines a candidate round key:

$$RK_{10,z}[b] = M_b \oplus SBox(z), \qquad z \in \{0, \ldots, 255\}. \quad (10)$$

The candidate equation follows directly from the AES final-round byte relation. With no MixColumns in the last round, write the output at byte b as

$$C_b = SBox(z_b) \oplus RK_{10}[b]. \quad (11)$$

For an absent ciphertext value M_b, substitute $C_b = M_b$ in (11) and XOR both sides with $SBox(z_b)$. This gives

$$RK_{10}[b] = M_b \oplus SBox(z_b).$$

The AES S-box is a permutation, so for a fixed round key and output byte there is exactly one corresponding input byte. The shared-endpoint hypothesis sets $z_b = z$ for all byte positions, yielding (10). Under the shared-z assumption, the 256 possible values of z give 256 candidate round keys. We use the power traces to rank these candidates.

For each candidate, the AES-128 schedule is reversed to obtain round keys, and standard AES states are reconstructed from the retained ciphertext rows. Predicted transition features are $H^{(z)}_{r,b}[j] = HW(S^{(z)}_{r-1}[j,b] \oplus S^{(z)}_r[j,b])$ for rounds 1-8 and retained trace j. At each detected event sample t_r, the code correlates this feature across traces with the measured sample vector. Its aggregate candidate score is

$$Q(z) = \frac{1}{8 \cdot 16} \sum_{r=1}^{8} \sum_{b=0}^{15} \left| \rho\Big(H^{(z)}_{r,b}, P(t_r)\Big) \right|. \quad (12)$$

The candidate with the largest $Q(z)$ is selected.

D. Task 4: Master-Key Recovery

We use Equation (6) to get the backward round keys from the recovered round-10 keys from Task 3.

The backward calculation still uses the forward S-box. We already know the input to F_i, so we can calculate its output using the usual rotation and substitution. The step that recovers the earlier word is the XOR in Equation (6). We therefore do not need to undo the S-box substitution when reversing the key schedule [6], [7].

The round constants follow the rounds in reverse order. At $i = 40, 36, \ldots, 4$, their first bytes are, in hexadecimal, `36, 1b, 80, 40, 20, 10, 08, 04, 02, 01` [6]. In `fourth_task.py`, the list of constants follows the forward order, and its first entry has index zero. The script therefore uses entry $i/4-1$ to select the constant for a backward step. For example, $i = 40$ selects entry 9, which contains `0x36`.

After the script reaches $w[0], \ldots, w[3]$, it joins those four words, keeping their original order, to form the master key candidate. To check the calculation, it runs the forward key schedule with this key and compares the resulting round-10 key with the input. A match confirms that the backward and forward calculations agree for the selected round key.

V. Results and Discussion

A. Task 1: Fake Round-10 Key

The full-data CPA selects the fake round-10 key

`91 de c0 68 e3 fa ce 45 99 ce 38 f4 20 86 3a 30.`

The winning peaks occur at sample 49 for most byte positions and at sample 50 for bytes 5 and 15. The per-byte peak absolute correlations range from approximately 0.119 to 0.178. Figure 2 shows byte 0 as an example; Appendix A contains the plots for bytes 1 through 15. The peak values are attack scores after searching 256 candidates over 200 samples, not statistical confidence bounds.

B. Task 2: Number of Traces

In the saved characterization, the first tested prefix with rank one for all 16 bytes contains 1,933 traces. Rank one is then observed for 100 consecutive tested counts beginning at 2,020. The earlier full-recovery point is a useful observed threshold for this ordering, while 2,020 is the more conservative stable threshold under the script's 100-count rule.

C. Task 3: Actual Round-10 Key

The relation in Equation (8) links each doubled ciphertext value to its Task 1 fake key byte through $SBox(0) = \texttt{0x63}$. We used this repeated pattern as evidence of a lookup-table artifact. It motivated us to test a related but separate assumption for the absent values: one missing S-box input z is shared across byte positions. This gives a manageable, testable set of 256 round-key candidates through Equation (10). However, it is not a mathematical consequence of the doubled-value relation and histogram alone cannot distinguish all candidates. Both $z = 0$ and $z = 0\text{xFF}$ remain possible at that stage.

The variance profile in Fig. 1 has ten selected local maxima at samples 13, 17, 21, 25, 29, 33, 37, 41, 45, and 49. Correlations for candidate transition features at the first eight event samples rank $z = 0\text{xFF}$ first. This gives the round-10 key candidate

`d0 14 f9 a8 c9 ee 25 89 e1 3f 0c c8 b6 63 0c a6.`

Figure 5 shows the candidate-correlation plots for byte 0 across rounds 1–8; Appendix B contains the plots for bytes 1 through 15. These plots help inspect whether candidate peaks occur near the detected event samples. The numerical selection criterion is the aggregate score in Equation (12).

D. Task 4: AES-128 Master Key

Starting with the round-10 key selected in Task 3, `fourth_task.py` applies Equation (6) until it reaches the first four words of the expanded key. Joining those words gives the master-key candidate

`2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c.`

The script checks this result by generating all the round keys again from the recovered master key. Its round-10 output is `d014f9a8c9ee2589e13f0cc8b6630ca6`, which matches the key supplied by Task 3.

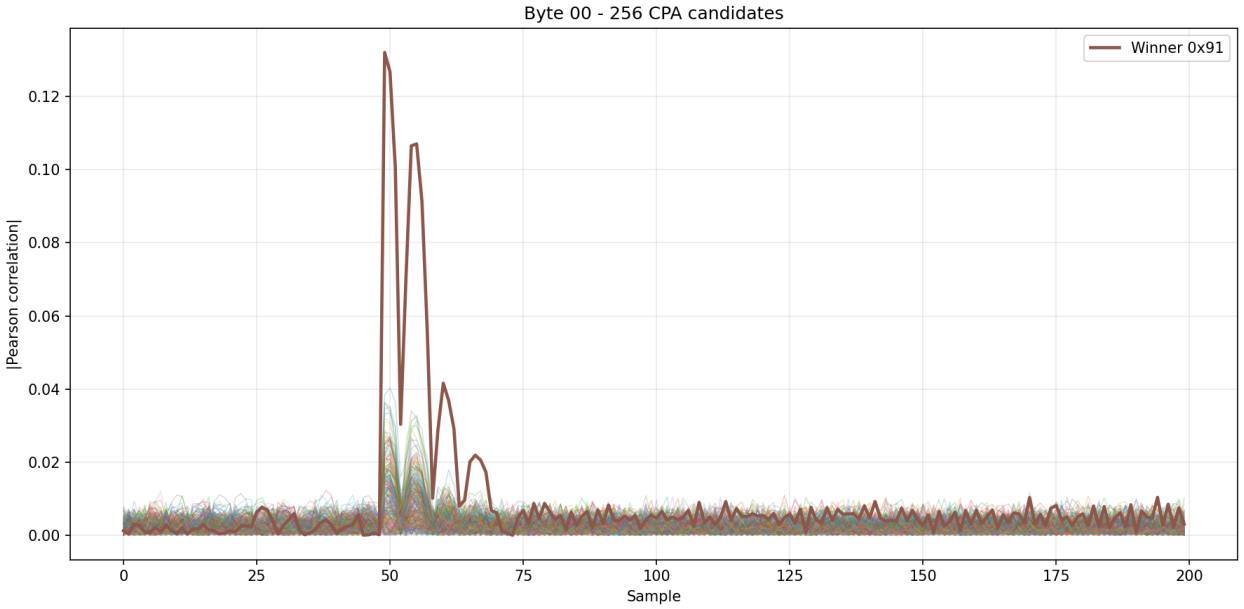


Fig. 2. Task 1 example: absolute CPA correlations for the 256 guesses at byte 0.

TABLE I
SUMMARY OF THE FOUR TASK RESULTS

Task	Target	Result
1	Fake round-10 key	`91dec068e3face45 99ce38f420863a30`
2	Trace count	First full recovery: 1,933; stable 100-count run begins: 2,020
3	Actual round-10 key	`d014f9a8c9ee2589 e13f0cc8b6630ca6`; selected $z = 0$xFF
4	AES-128 master key	`2b7e151628aed2a6 abf7158809cf4f3c`

This check confirms that the recovered master key corresponds to the selected round-10 key. Since possible 128-bit round key leads to a corresponding AES-128 master key when we reverse the schedule, we therefore use the trace analysis in Task 3 to support the key selection, and the forward expansion check in Task 4 to support the calculation.

VI. Limitations

Task 2 conclusion is dependent on the random seed, which is included to counter the ordering of traces. If the seed is changed the minimum number of traces might differ.

Task 3 conclusion depends on several assumptions: the same missing S-box input applies to all 16 byte positions; retained rows correspond to standard AES encryptions; the selected variance peaks correspond to AES round events; and ordinary-round leakage is sufficiently described by transition Hamming distance.

Task 4 starts with the round-10 key that Task 3 selects for $z =$ `0xFF`. Once we choose that round key, the backward calculation gives one corresponding master key. Expanding it forward checks that we have performed the key-schedule calculation correctly. The evidence that this is the device's actual key still comes from Task 3 and depends on the assumptions listed above.

VII. Conclusion

CPA over the final-round transition model recovers the fake round-10 key, and a rank characteristics shows how that reference key performs as the trace count grows. Ciphertext-frequency observations then provide a set of actual round-key candidates, while earlier-round transition correlations favor the shared endpoint $z = 0$xFF. Reversing the calculated RK10 derived from z values gives the corresponding master-key candidate. The fake key result follows directly from the stated CPA ranking; the actual-key result is shown probable by using correlation plots.

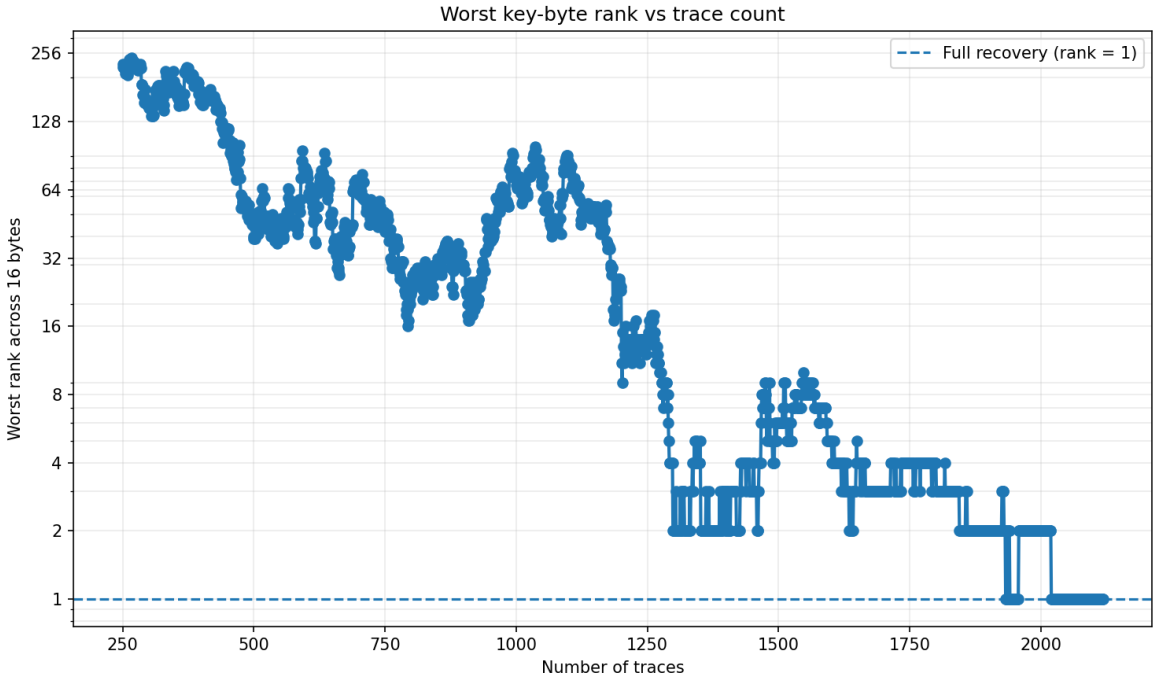


Fig. 3. Worst reference-key-byte rank as the number of traces increases. Rank one means all 16 reference bytes rank first.

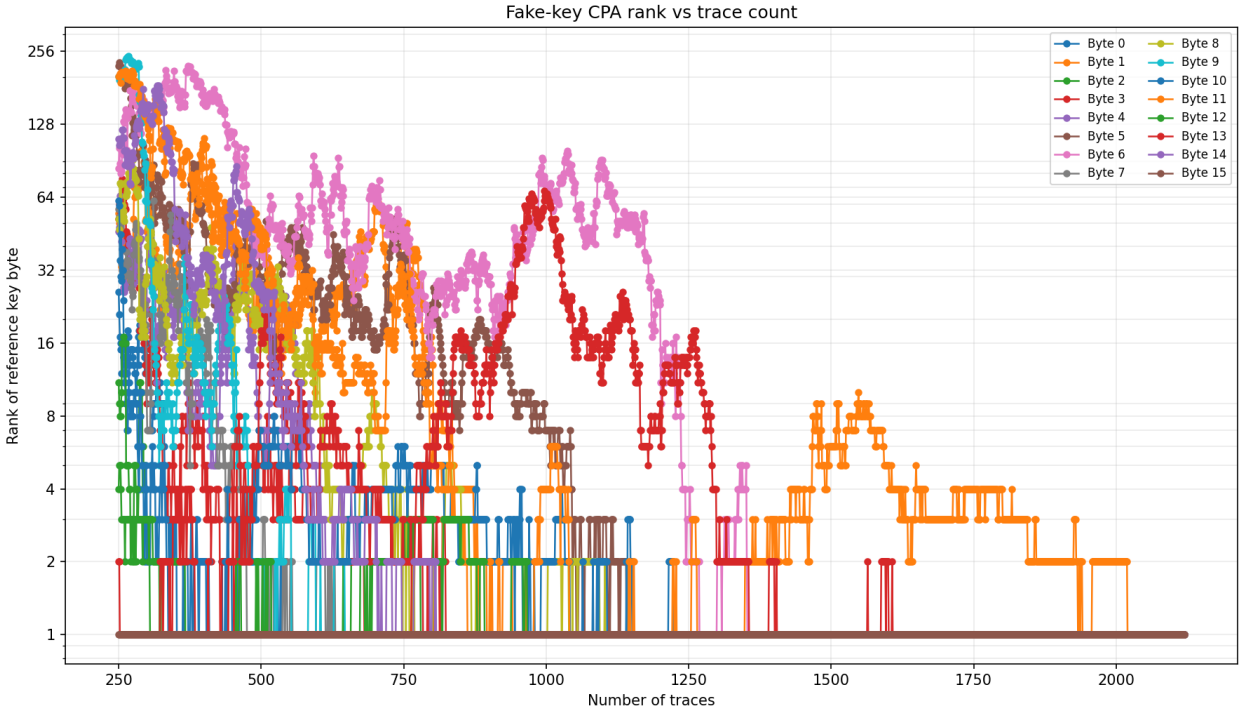


Fig. 4. Reference-key-byte ranks across the tested trace counts.

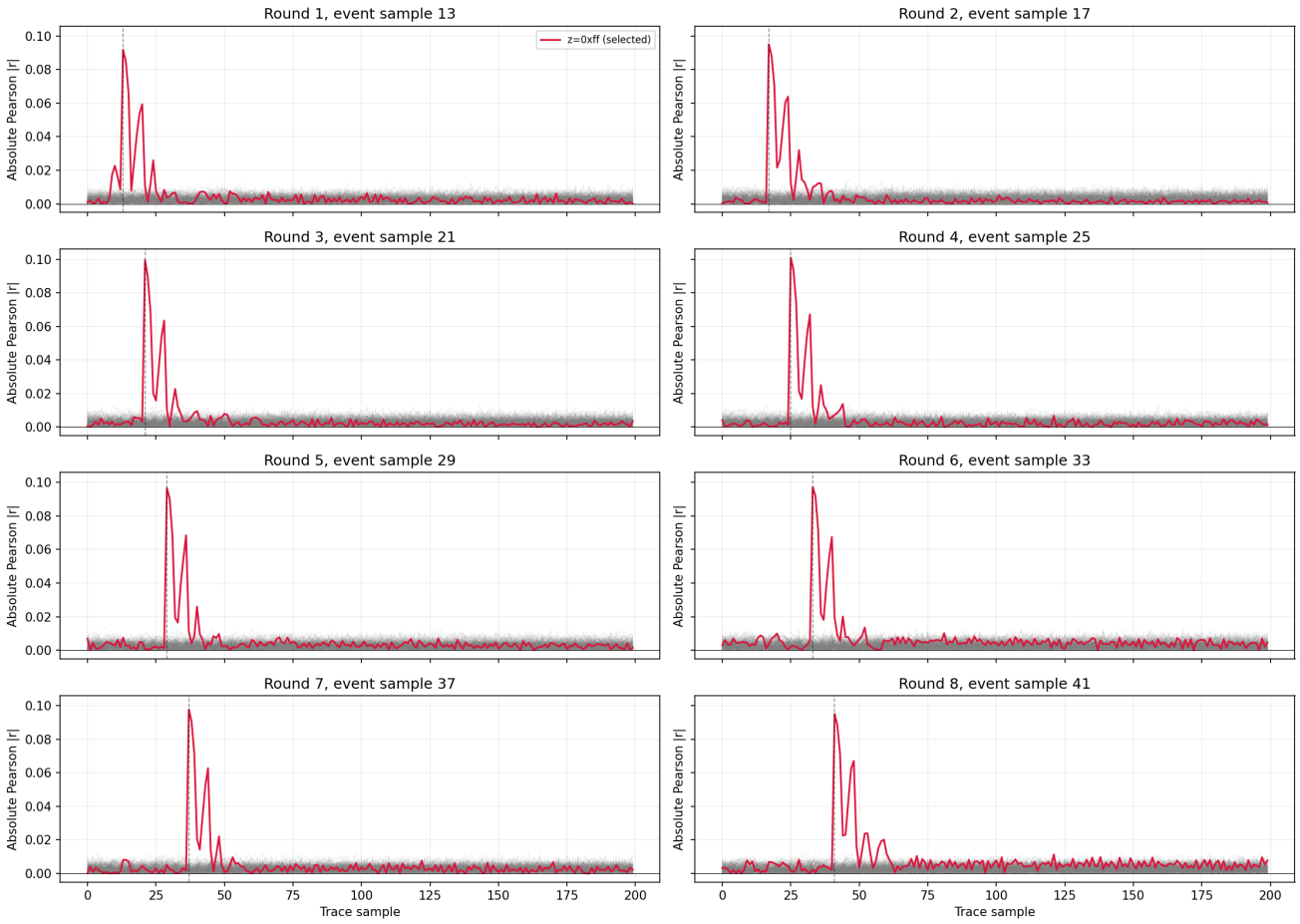


Fig. 5. Task 3 example: endpoint-candidate correlations for byte 0 across rounds 1–8.

Appendix A
Fake round-10 key correlation plots

Figure 2 shows byte 0. The following plots cover bytes 1 through 15.

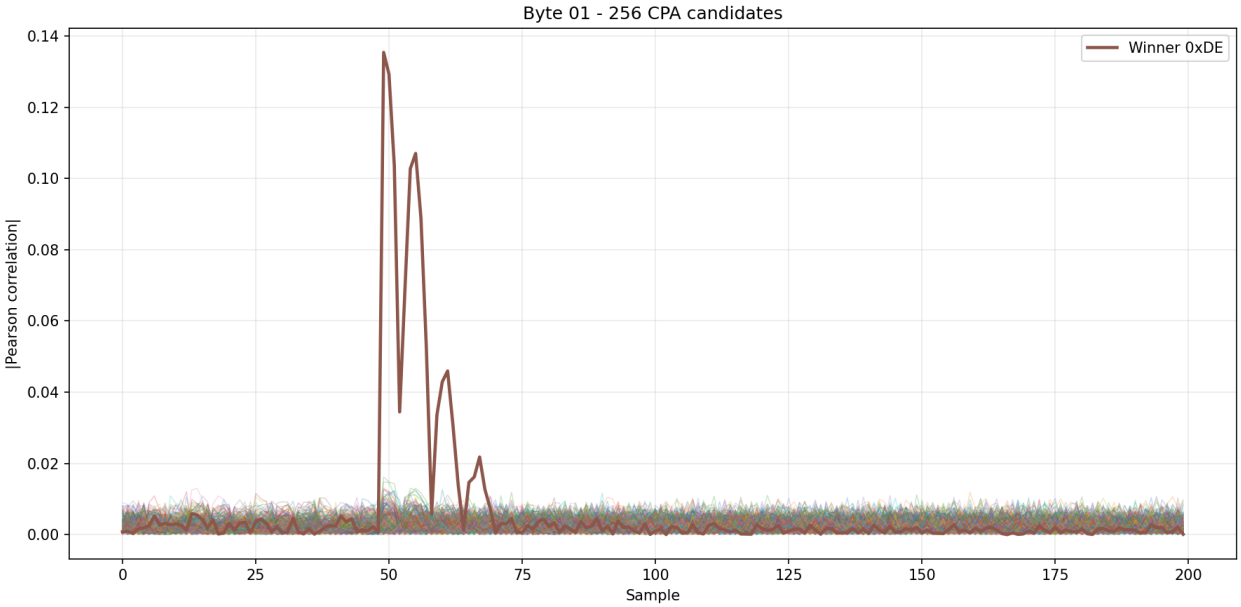


Fig. 6. Task 1: absolute CPA correlations for all 256 candidates at byte 01. The thick curve marks the winning candidate.

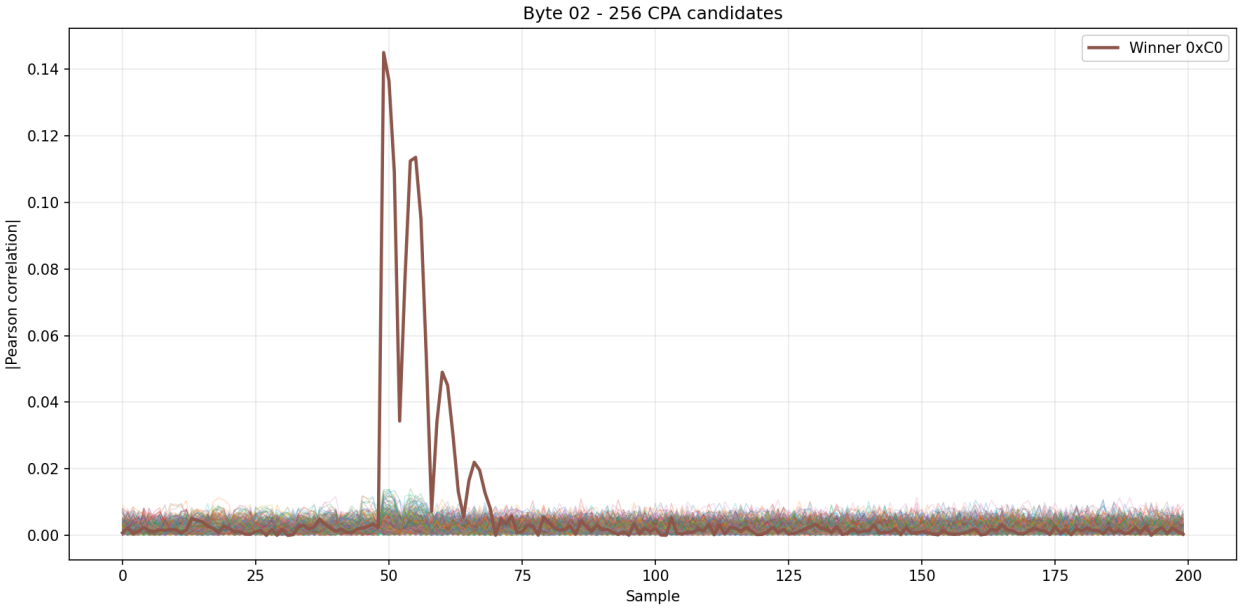


Fig. 7. Task 1: absolute CPA correlations for all 256 candidates at byte 02. The thick curve marks the winning candidate.

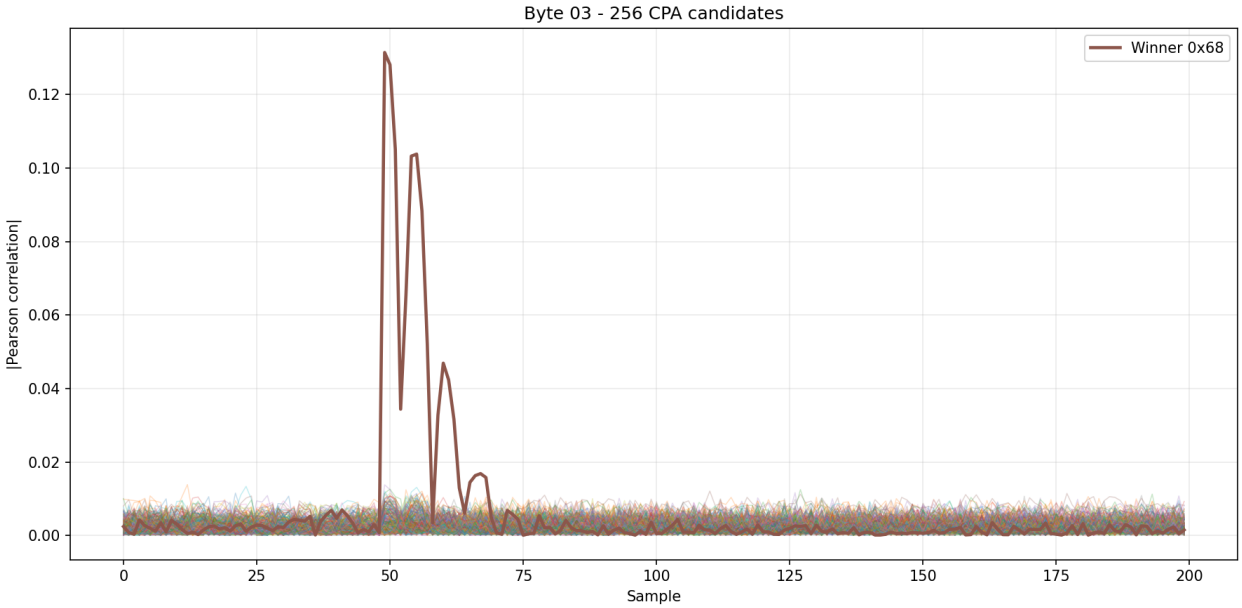


Fig. 8. Task 1: absolute CPA correlations for all 256 candidates at byte 03. The thick curve marks the winning candidate.

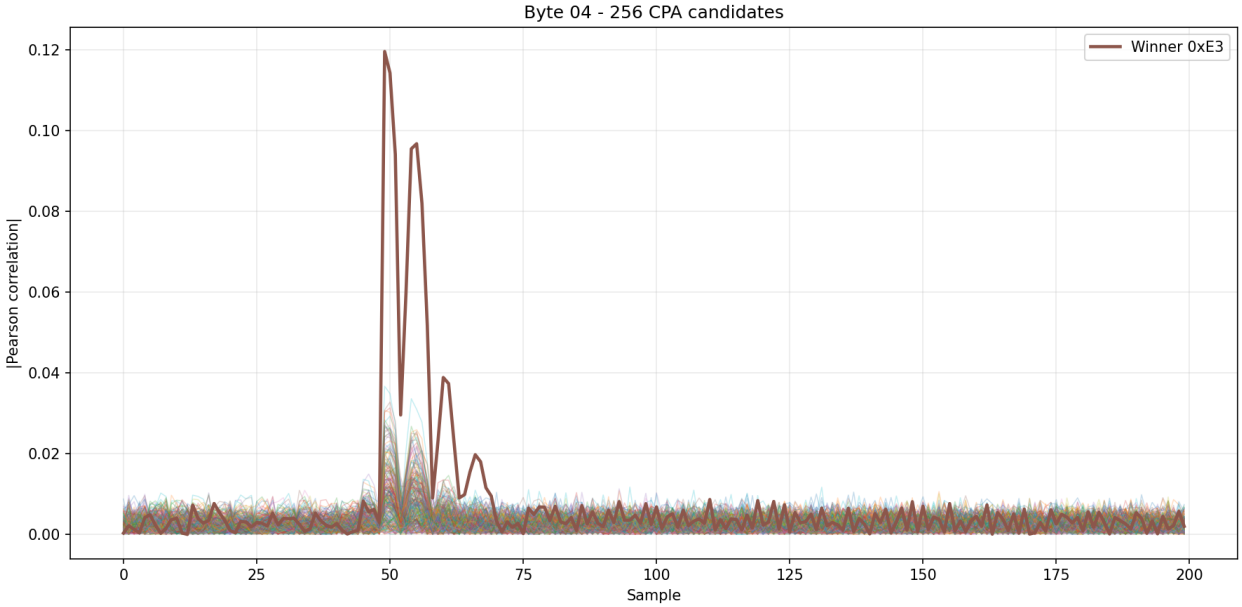


Fig. 9. Task 1: absolute CPA correlations for all 256 candidates at byte 04. The thick curve marks the winning candidate.

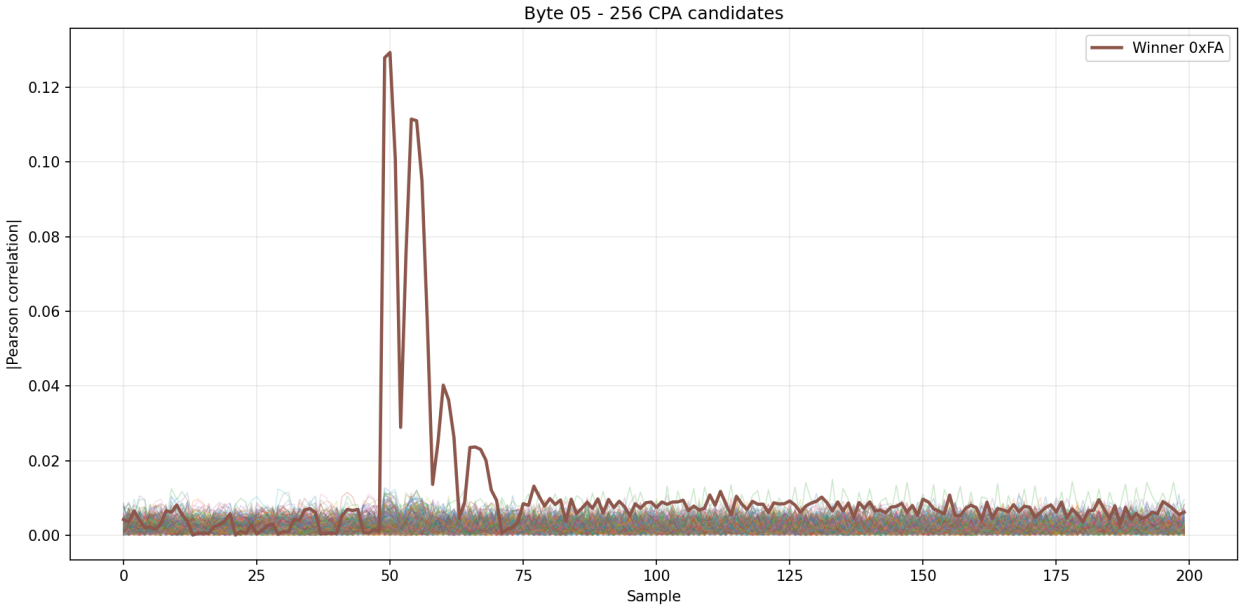


Fig. 10. Task 1: absolute CPA correlations for all 256 candidates at byte 05. The thick curve marks the winning candidate.

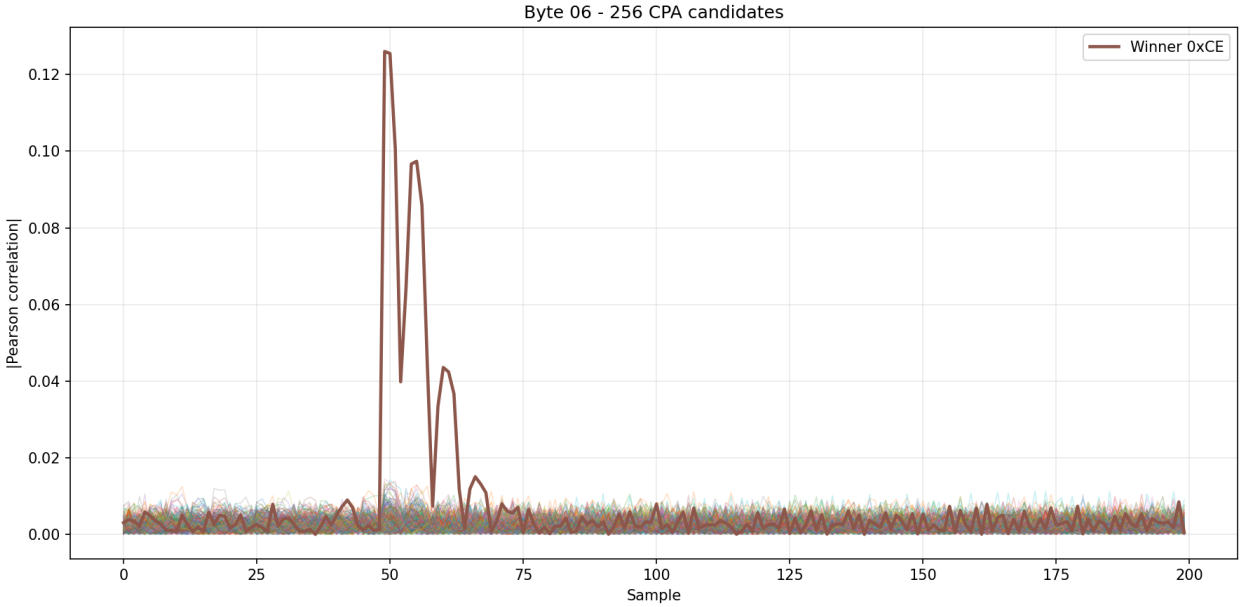


Fig. 11. Task 1: absolute CPA correlations for all 256 candidates at byte 06. The thick curve marks the winning candidate.

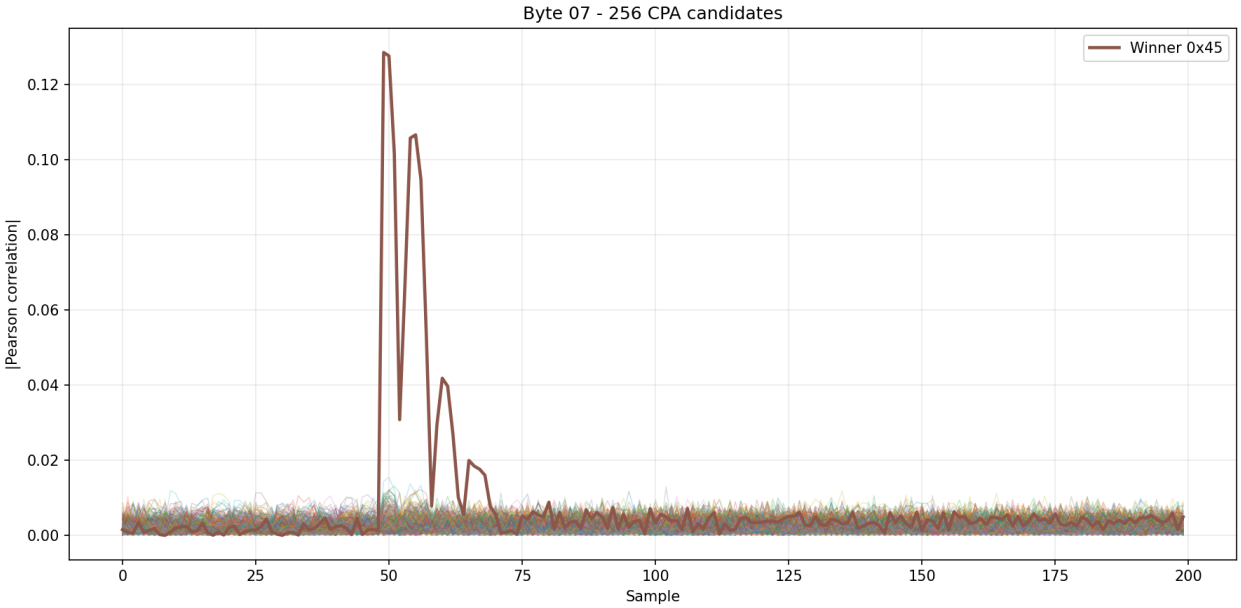


Fig. 12. Task 1: absolute CPA correlations for all 256 candidates at byte 07. The thick curve marks the winning candidate.

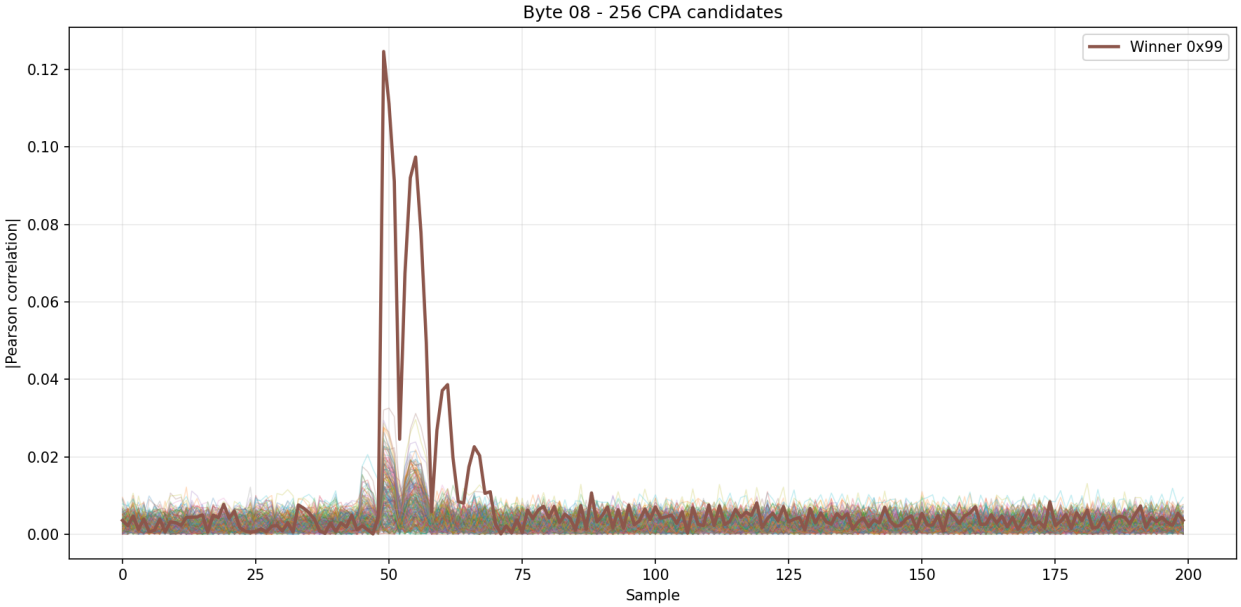


Fig. 13. Task 1: absolute CPA correlations for all 256 candidates at byte 08. The thick curve marks the winning candidate.

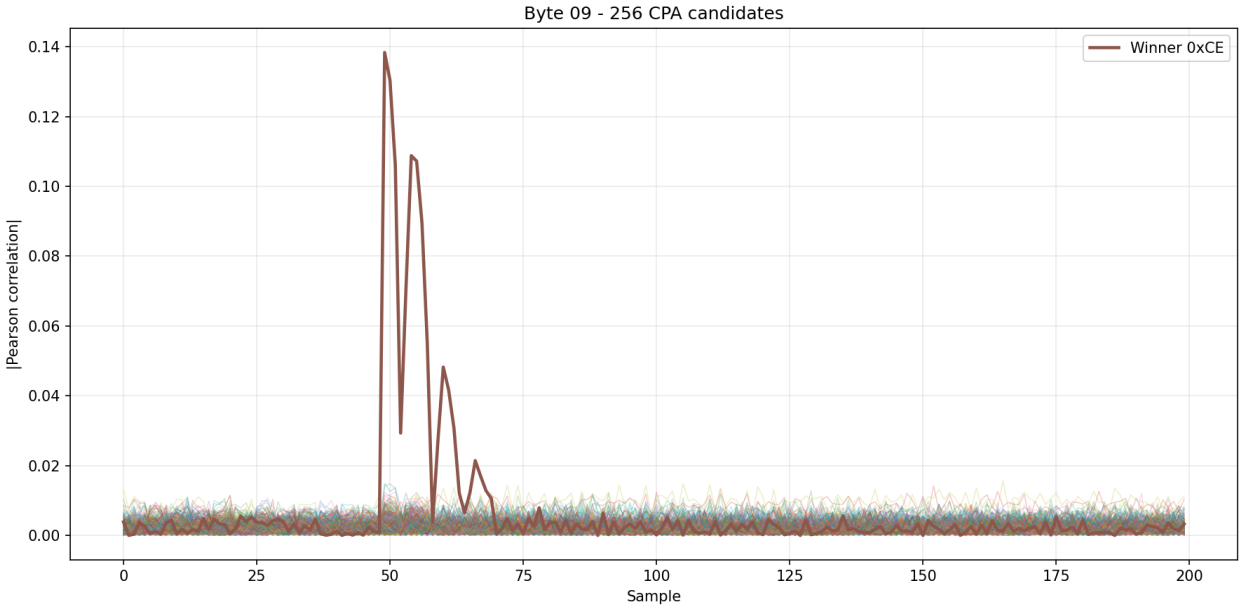


Fig. 14. Task 1: absolute CPA correlations for all 256 candidates at byte 09. The thick curve marks the winning candidate.

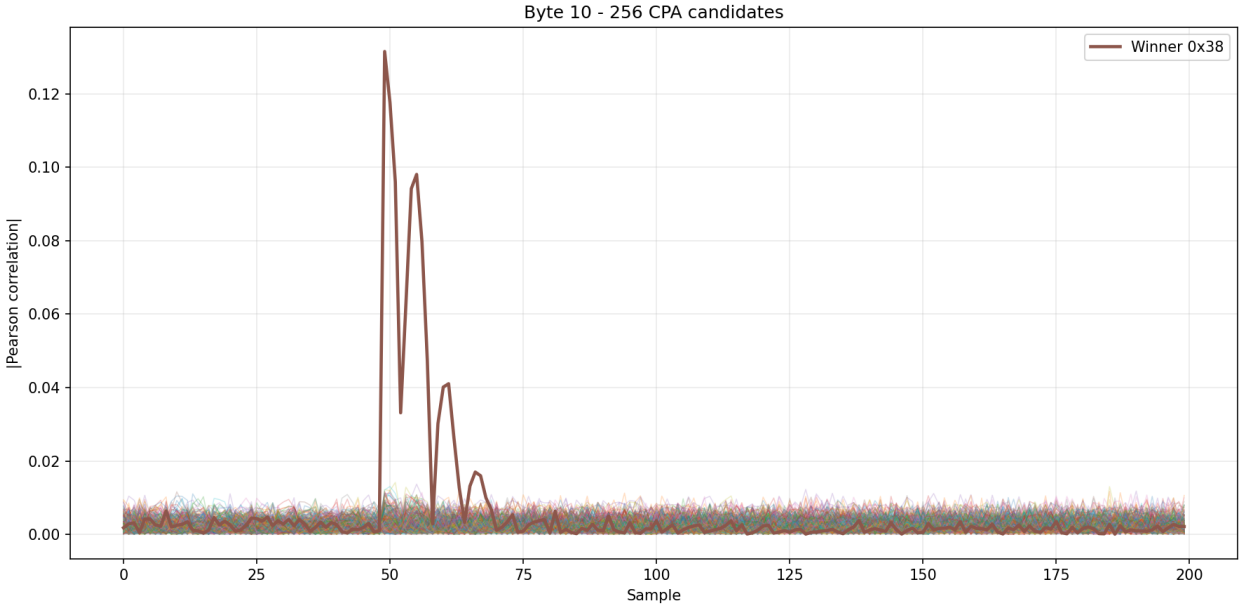


Fig. 15. Task 1: absolute CPA correlations for all 256 candidates at byte 10. The thick curve marks the winning candidate.

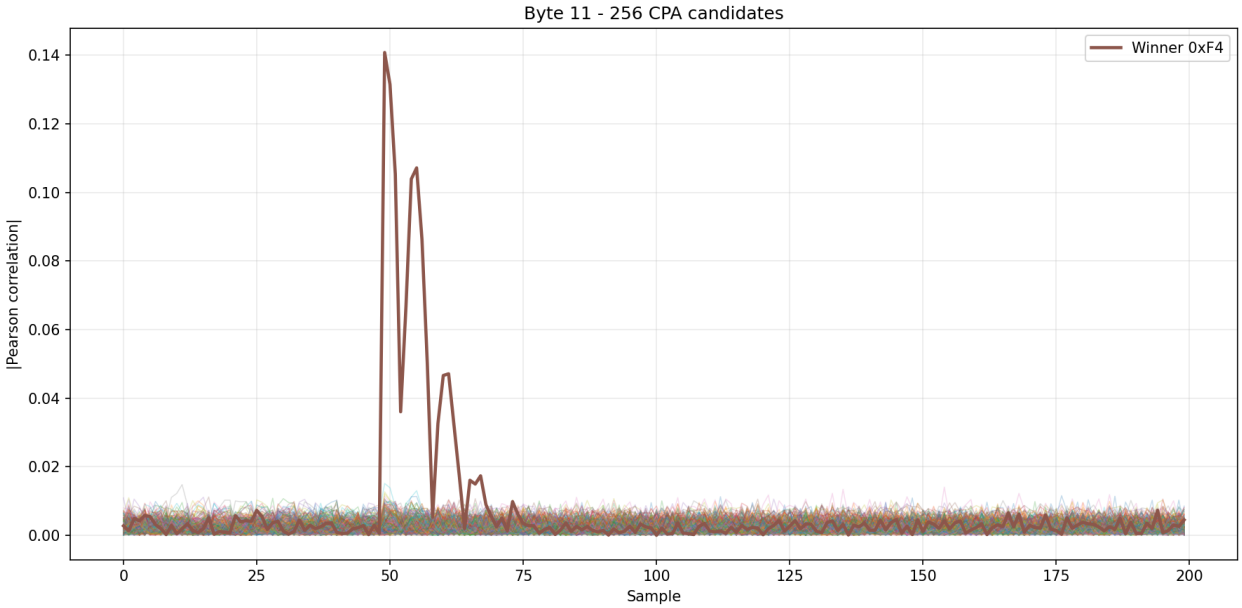


Fig. 16. Task 1: absolute CPA correlations for all 256 candidates at byte 11. The thick curve marks the winning candidate.

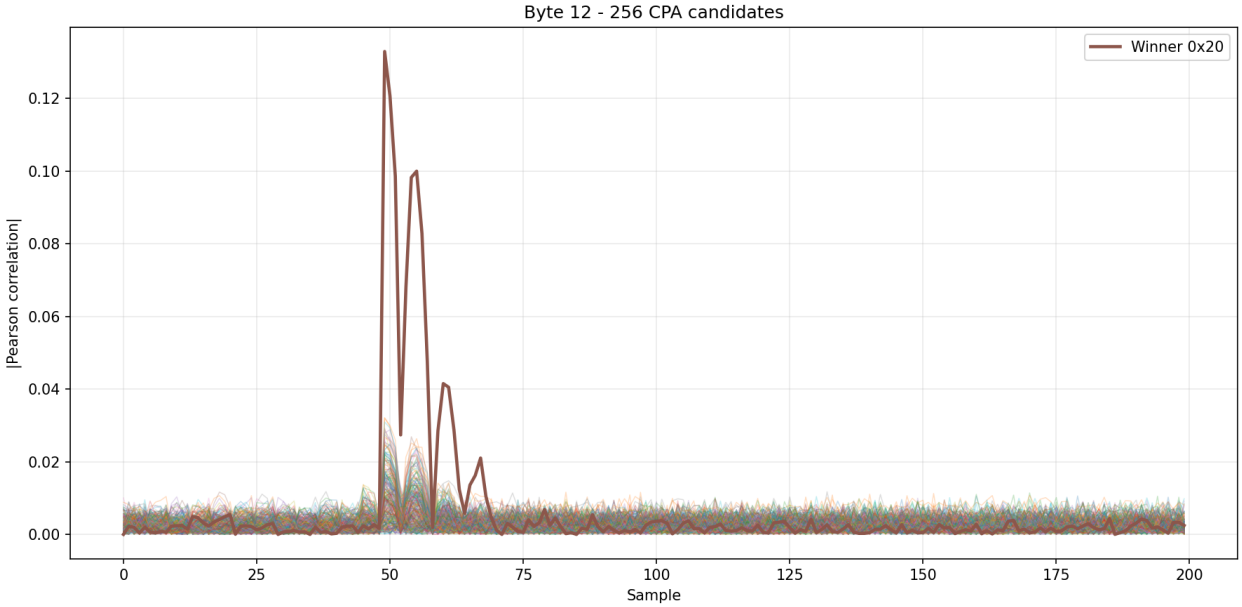


Fig. 17. Task 1: absolute CPA correlations for all 256 candidates at byte 12. The thick curve marks the winning candidate.

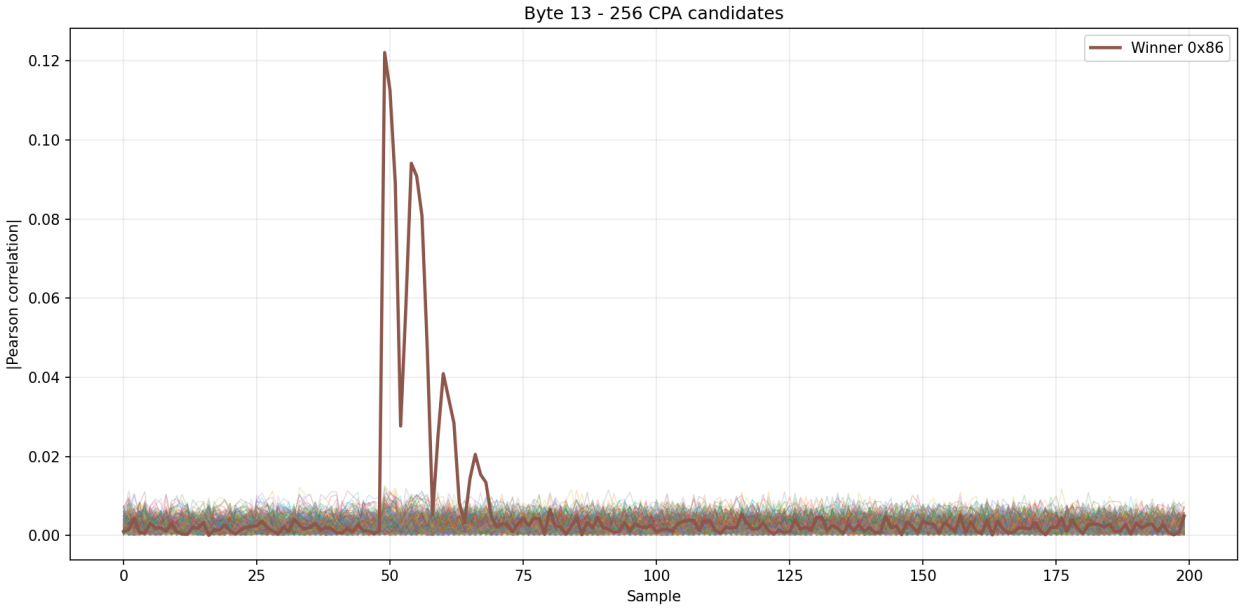


Fig. 18. Task 1: absolute CPA correlations for all 256 candidates at byte 13. The thick curve marks the winning candidate.

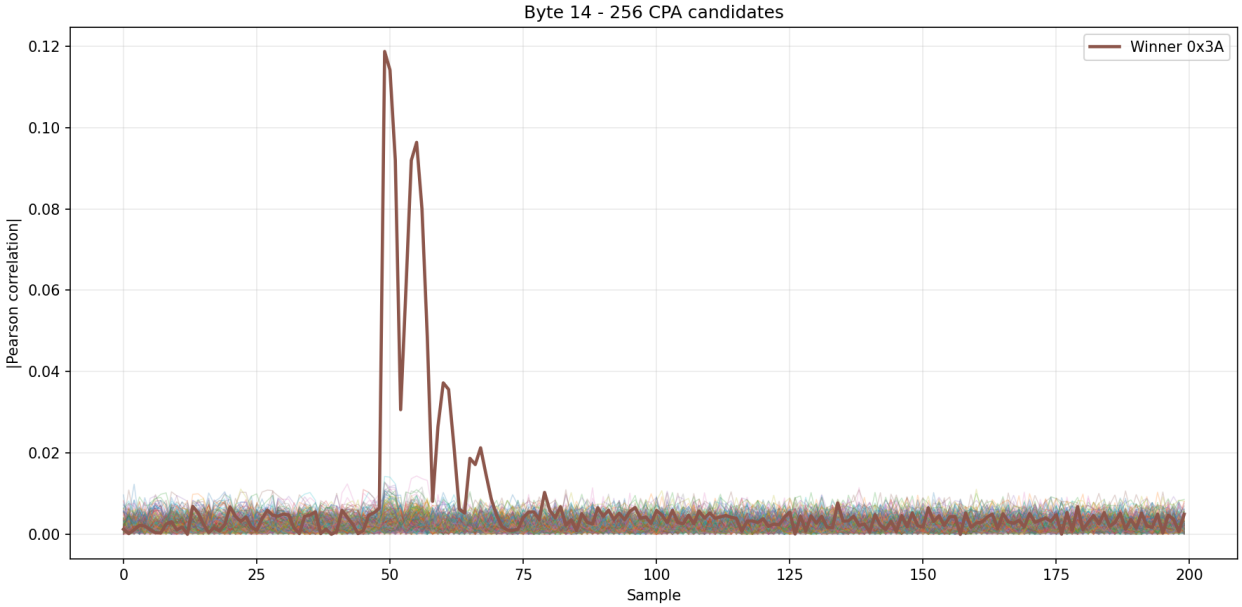


Fig. 19. Task 1: absolute CPA correlations for all 256 candidates at byte 14. The thick curve marks the winning candidate.

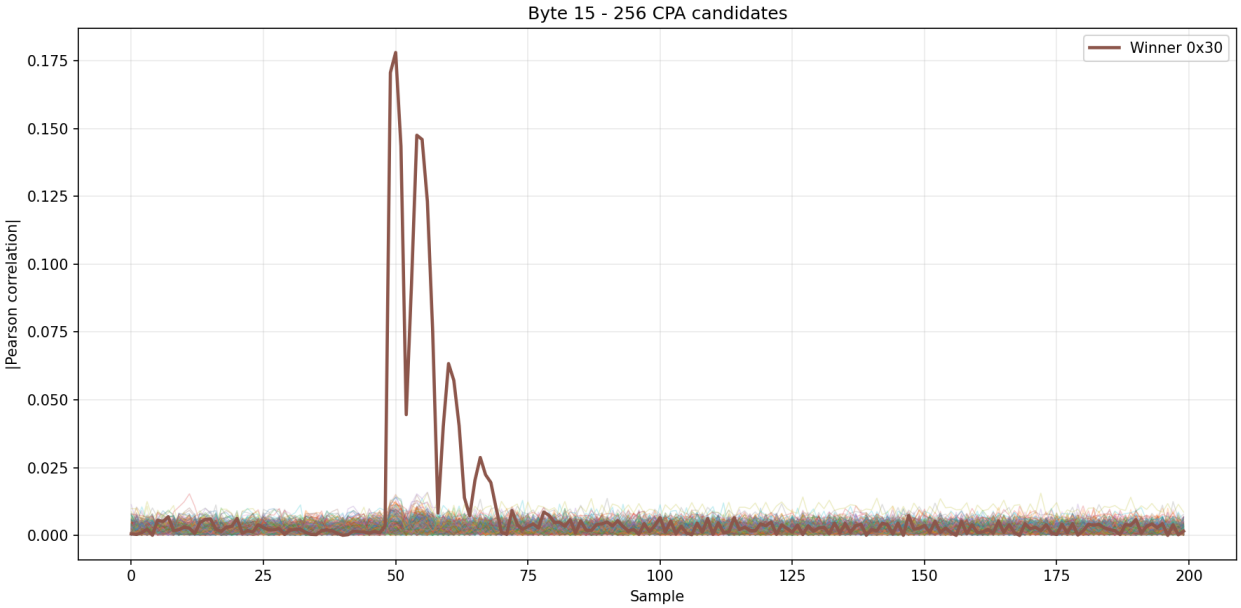


Fig. 20. Task 1: absolute CPA correlations for all 256 candidates at byte 15. The thick curve marks the winning candidate.

Appendix B
Actual round-10 key correlation plots

Figure 5 shows byte 0. The following plots cover bytes 1 through 15.

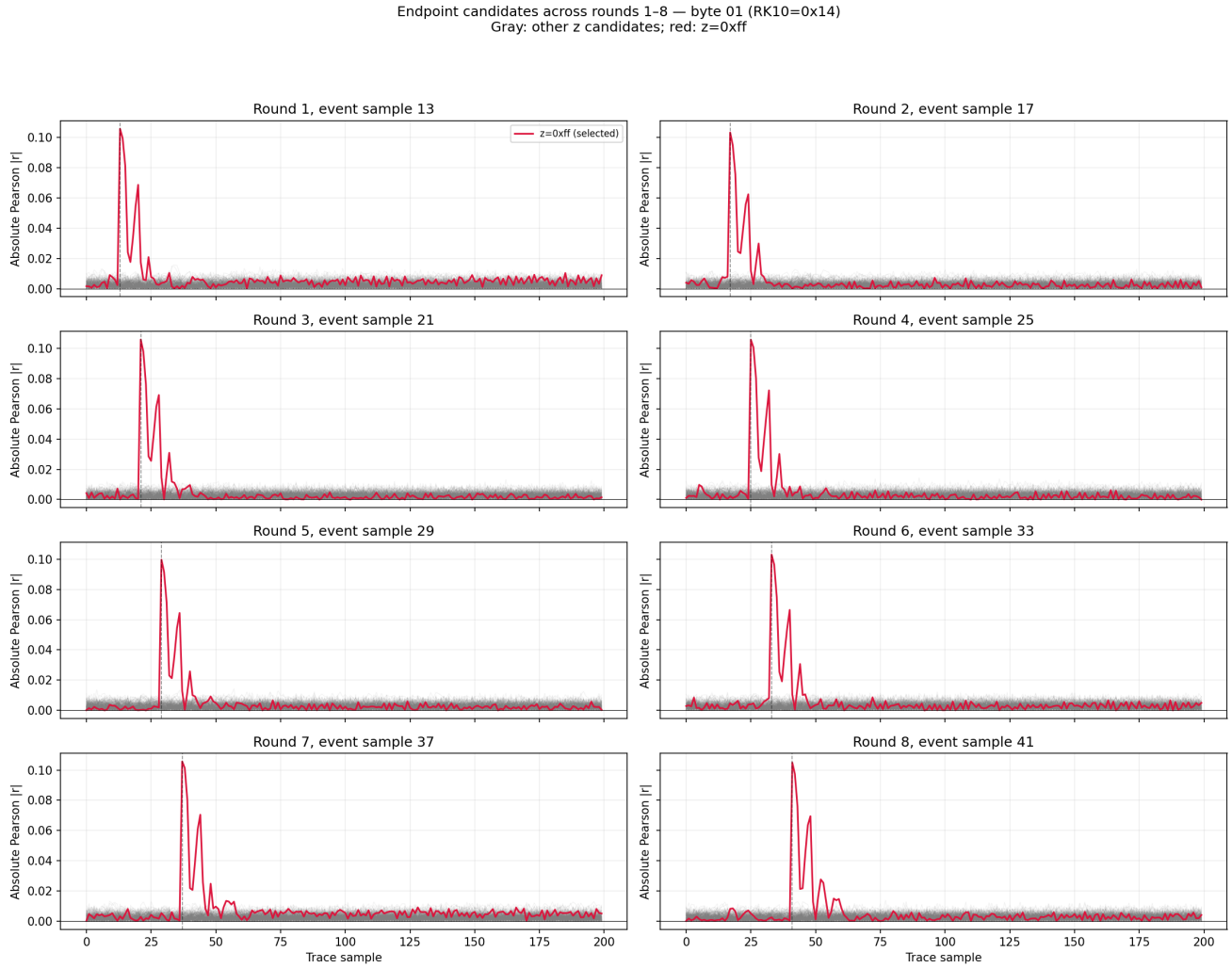


Fig. 21. Task 3: endpoint-candidate correlations for byte 01 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

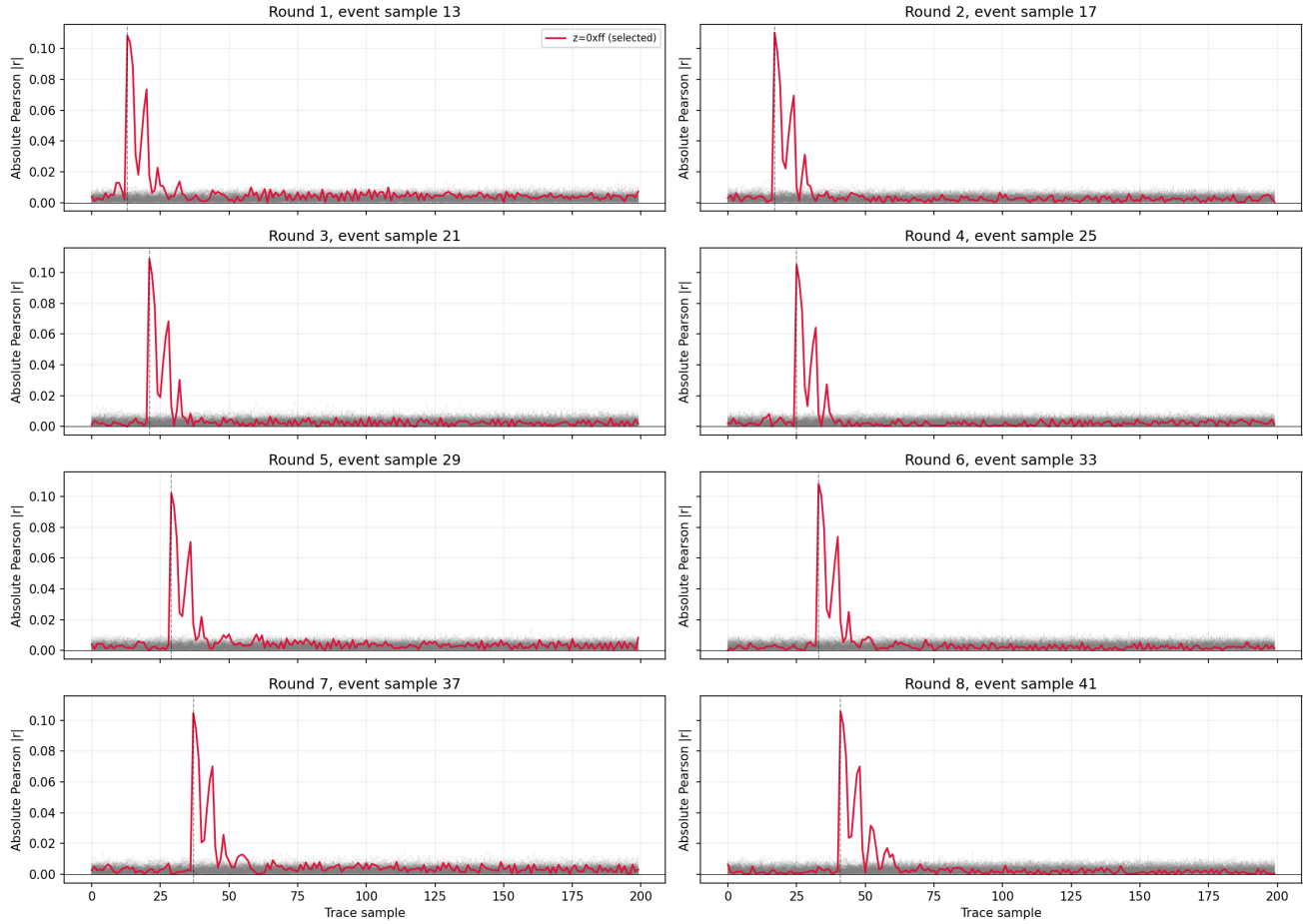


Fig. 22. Task 3: endpoint-candidate correlations for byte 02 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

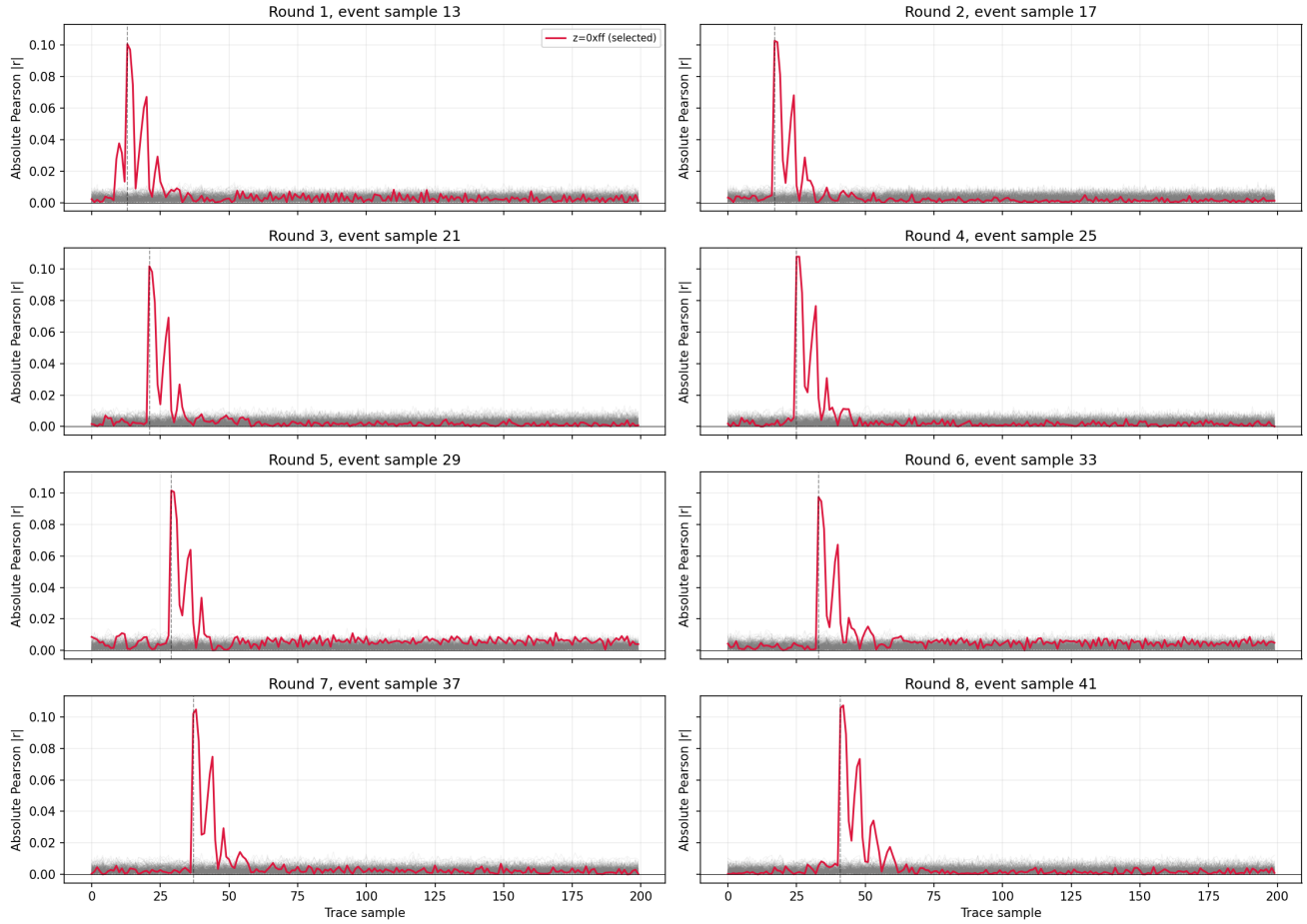


Fig. 23. Task 3: endpoint-candidate correlations for byte 03 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

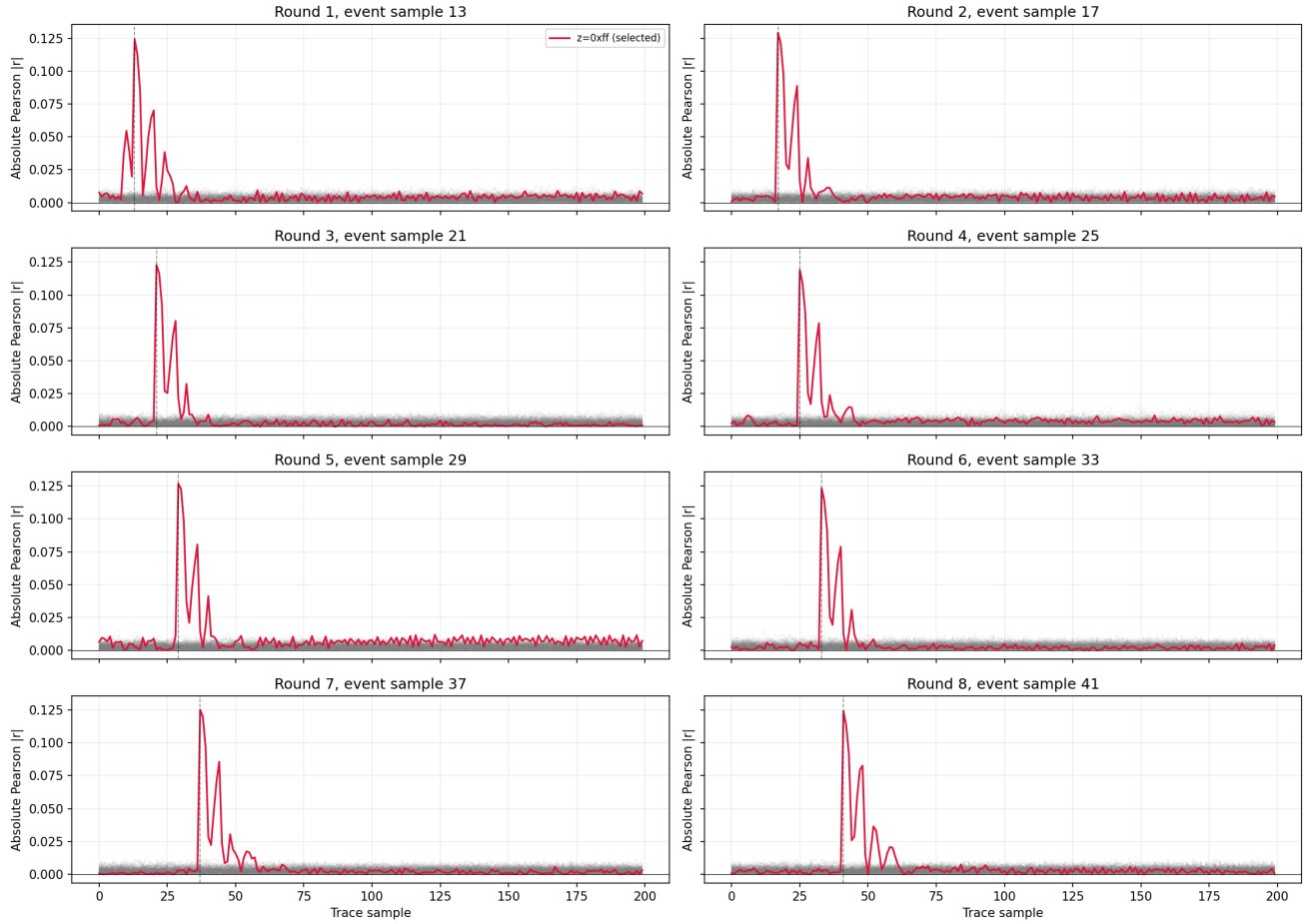


Fig. 24. Task 3: endpoint-candidate correlations for byte 04 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

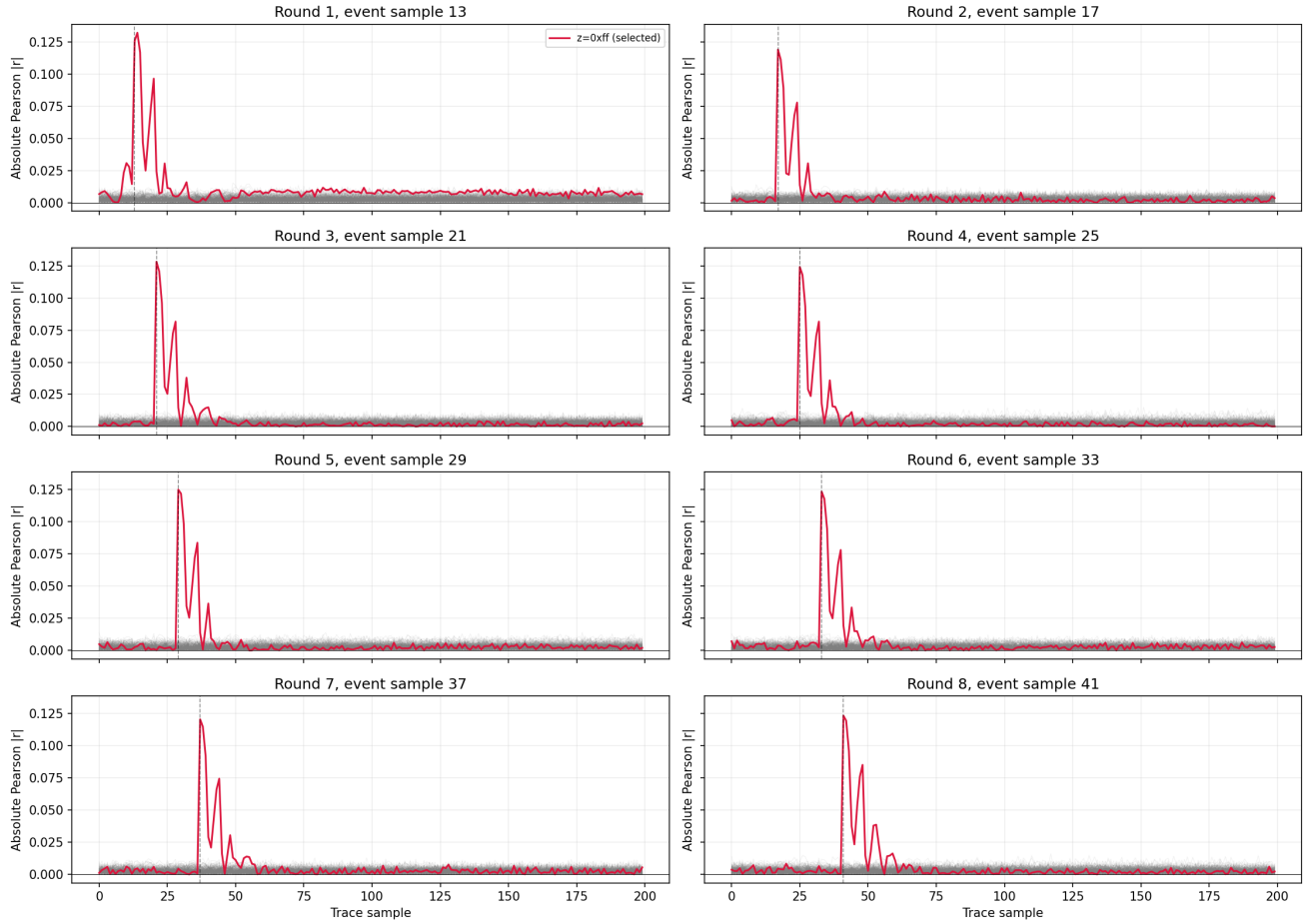


Fig. 25. Task 3: endpoint-candidate correlations for byte 05 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

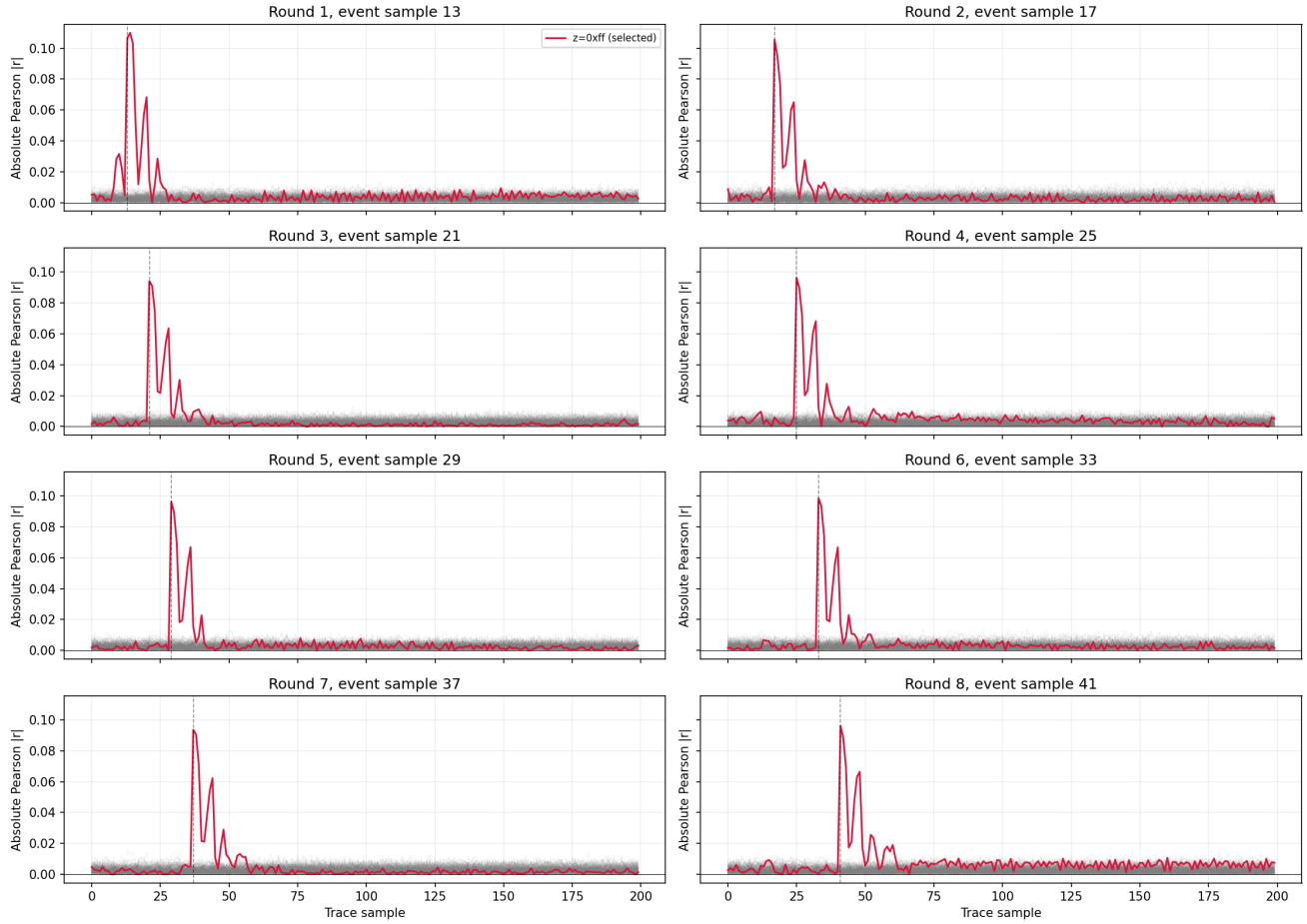


Fig. 26. Task 3: endpoint-candidate correlations for byte 06 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

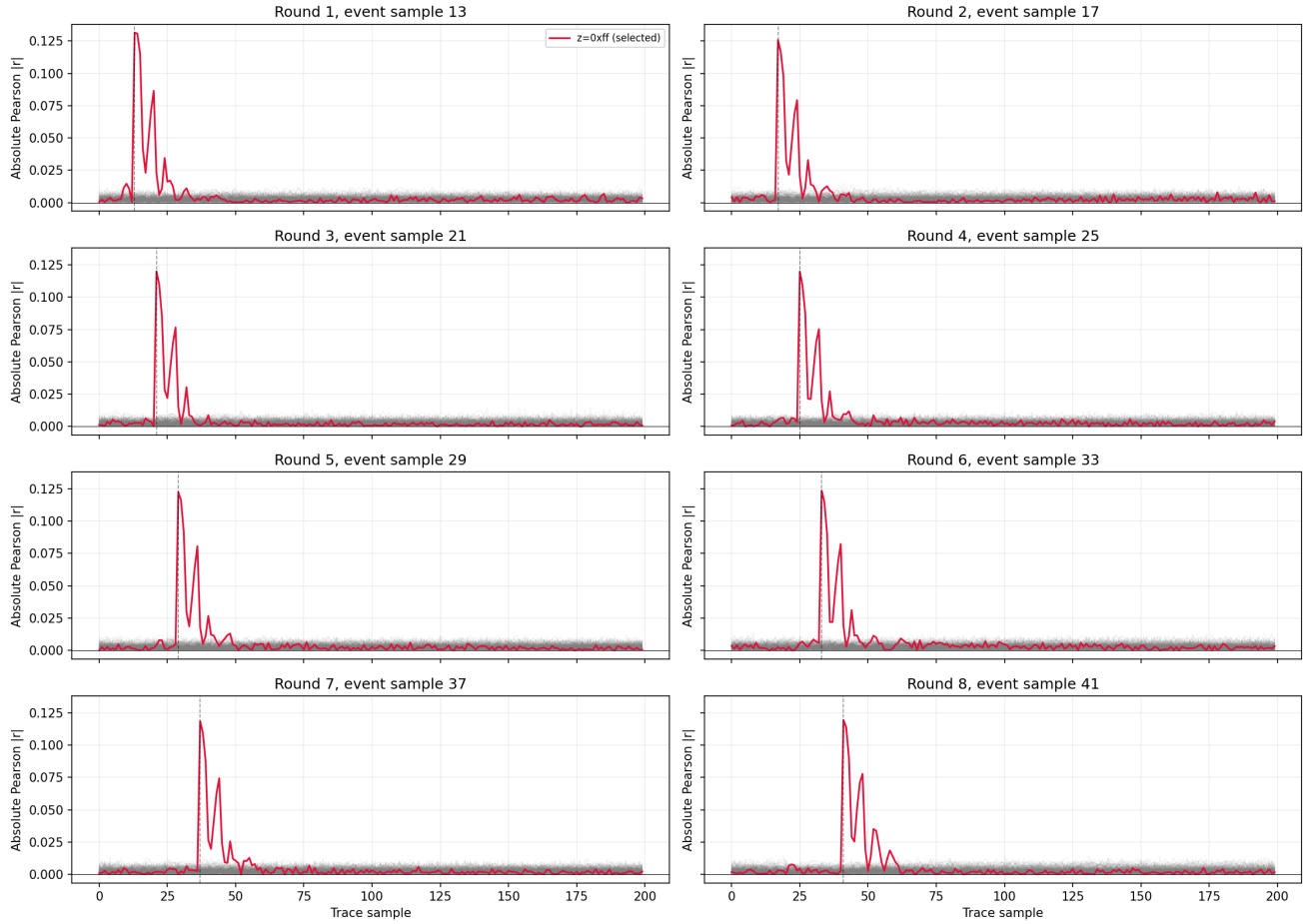


Fig. 27. Task 3: endpoint-candidate correlations for byte 07 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

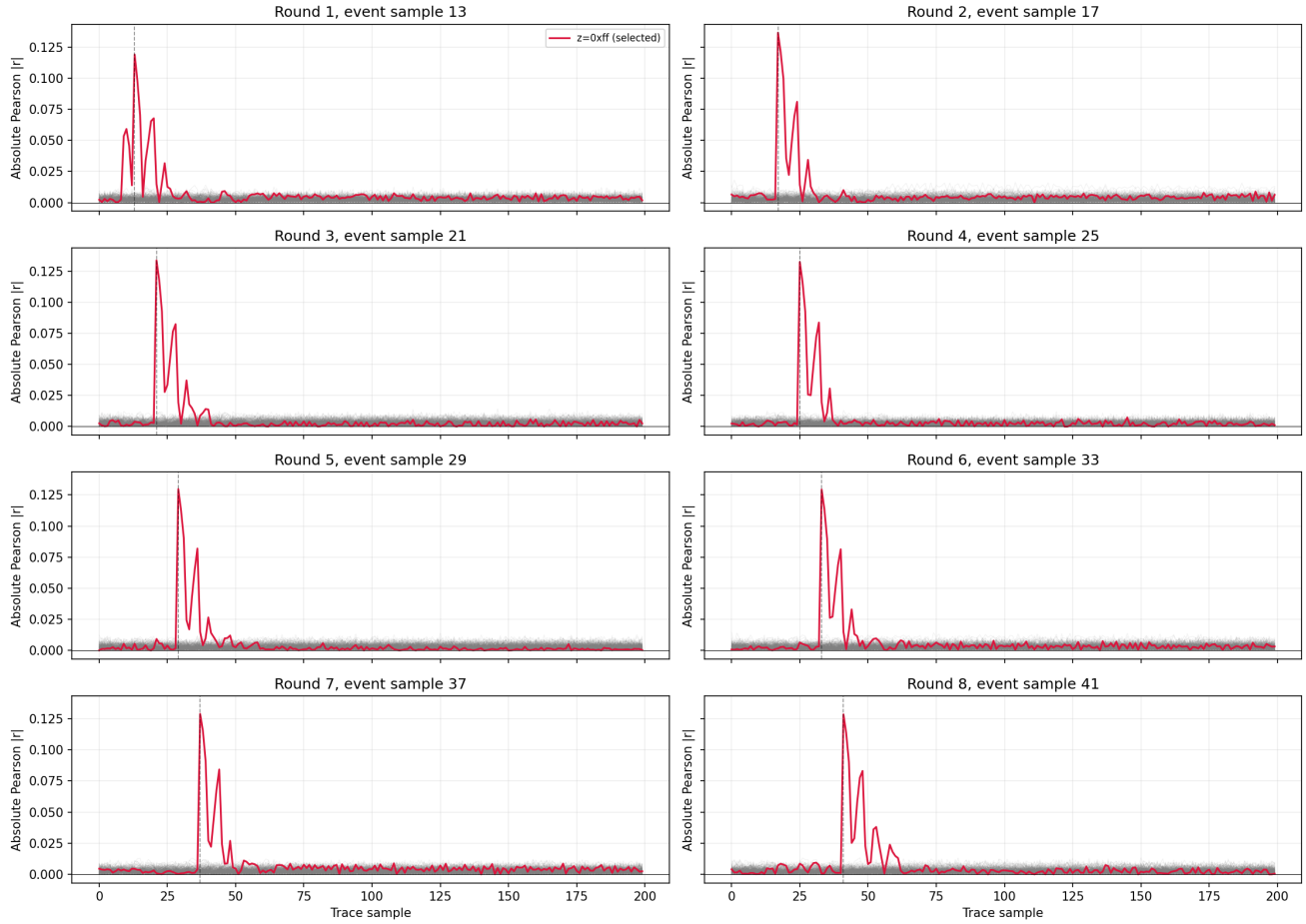


Fig. 28. Task 3: endpoint-candidate correlations for byte 08 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

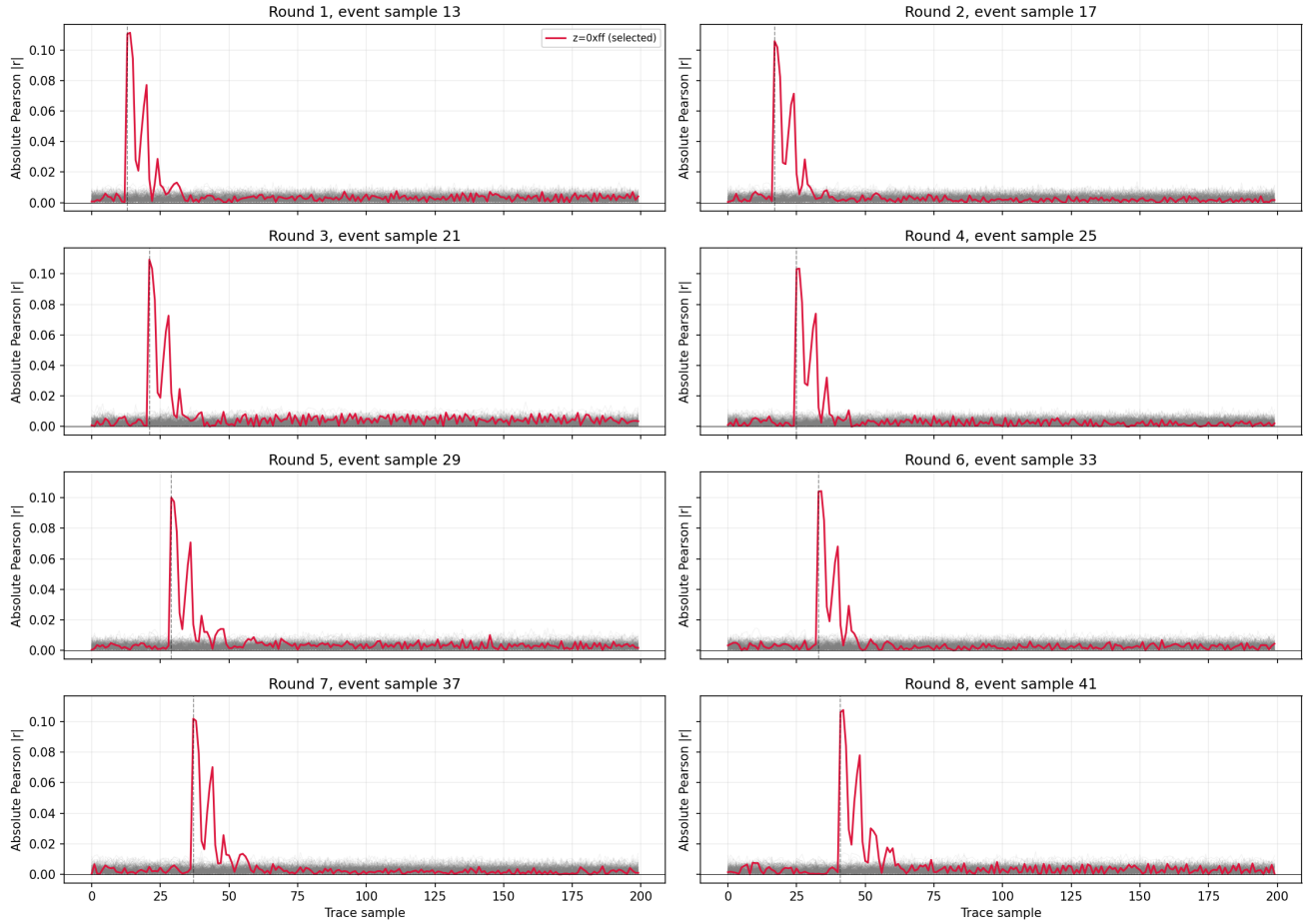


Fig. 29. Task 3: endpoint-candidate correlations for byte 09 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

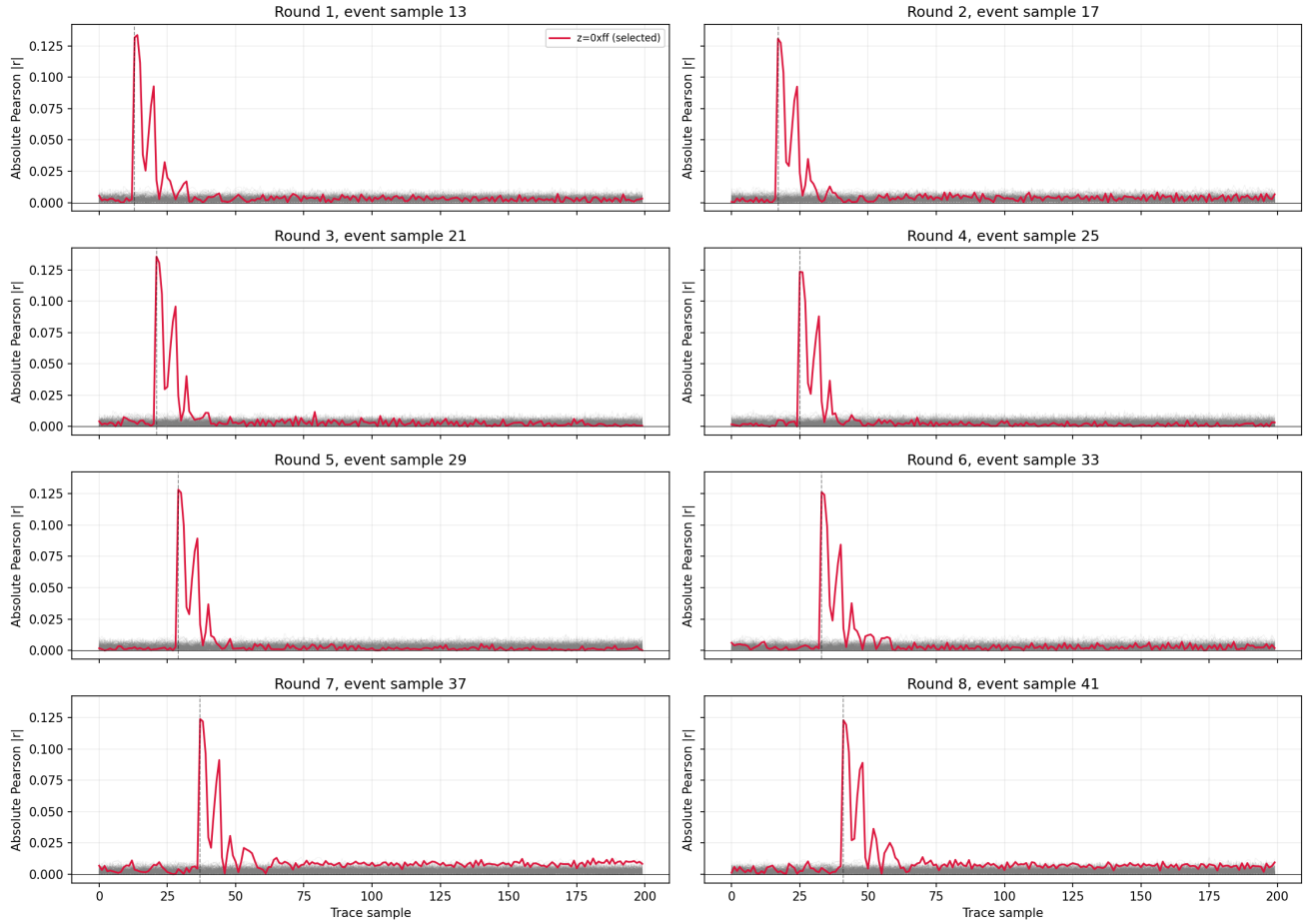


Fig. 30. Task 3: endpoint-candidate correlations for byte 10 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

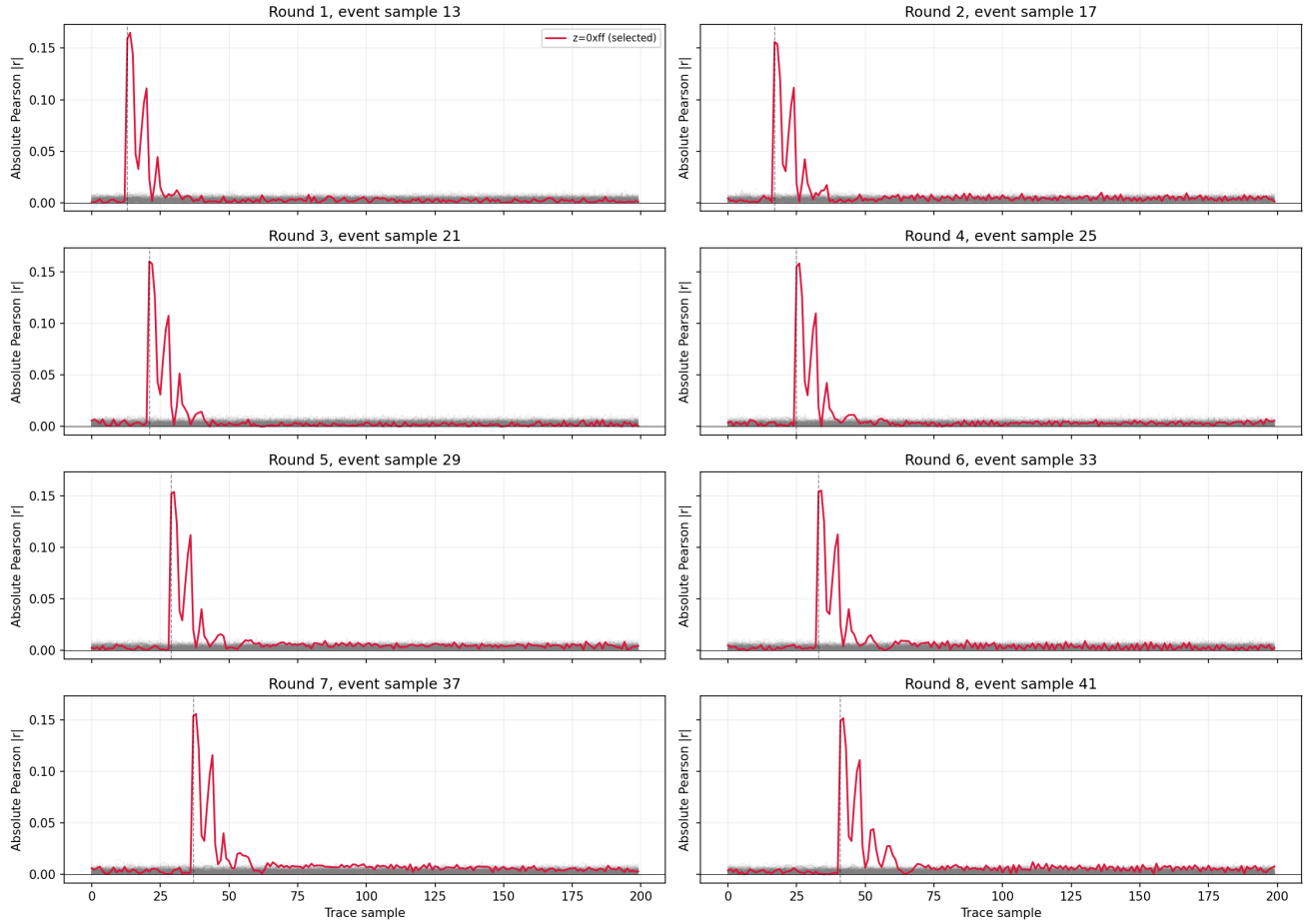


Fig. 31. Task 3: endpoint-candidate correlations for byte 11 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

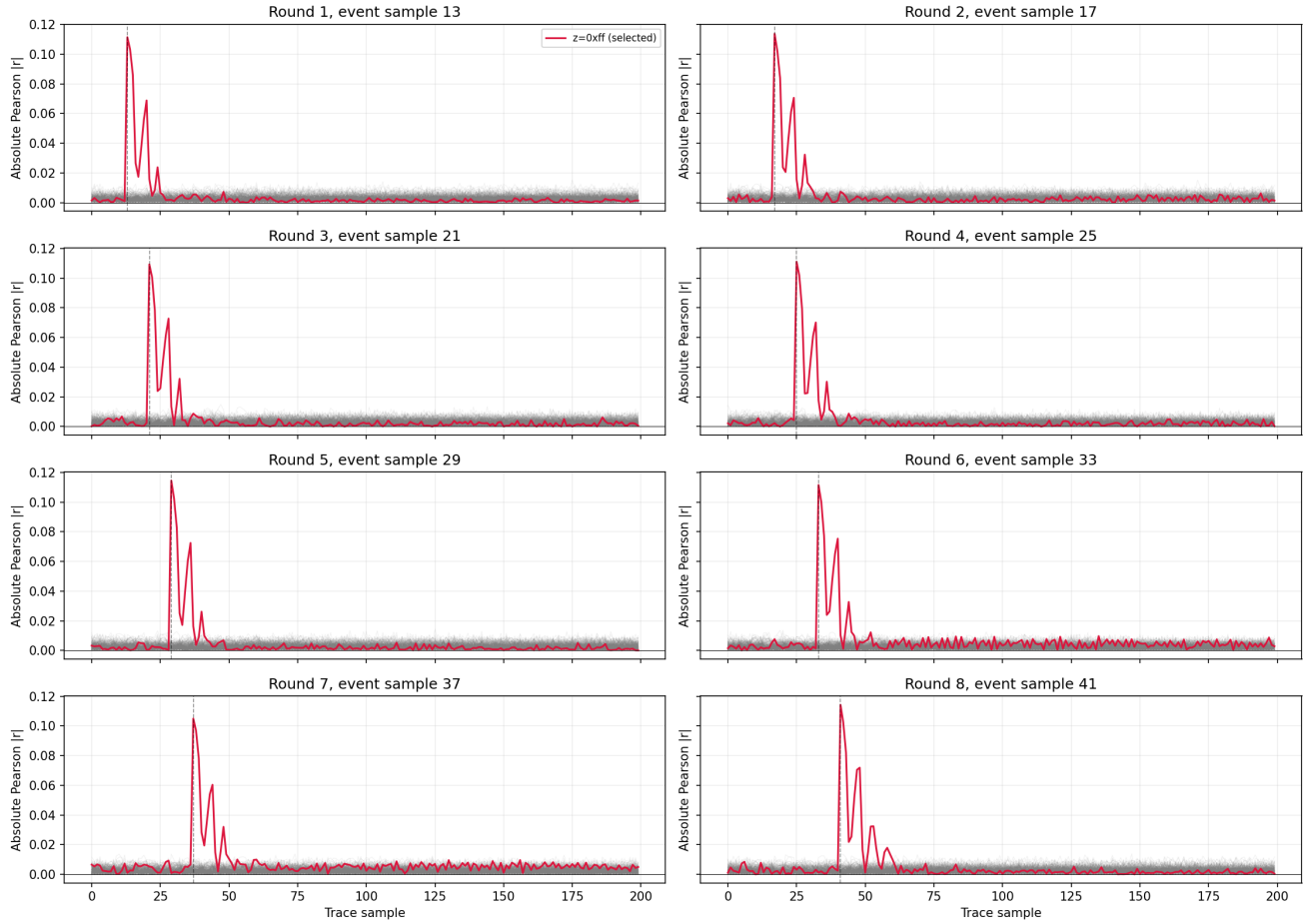


Fig. 32. Task 3: endpoint-candidate correlations for byte 12 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

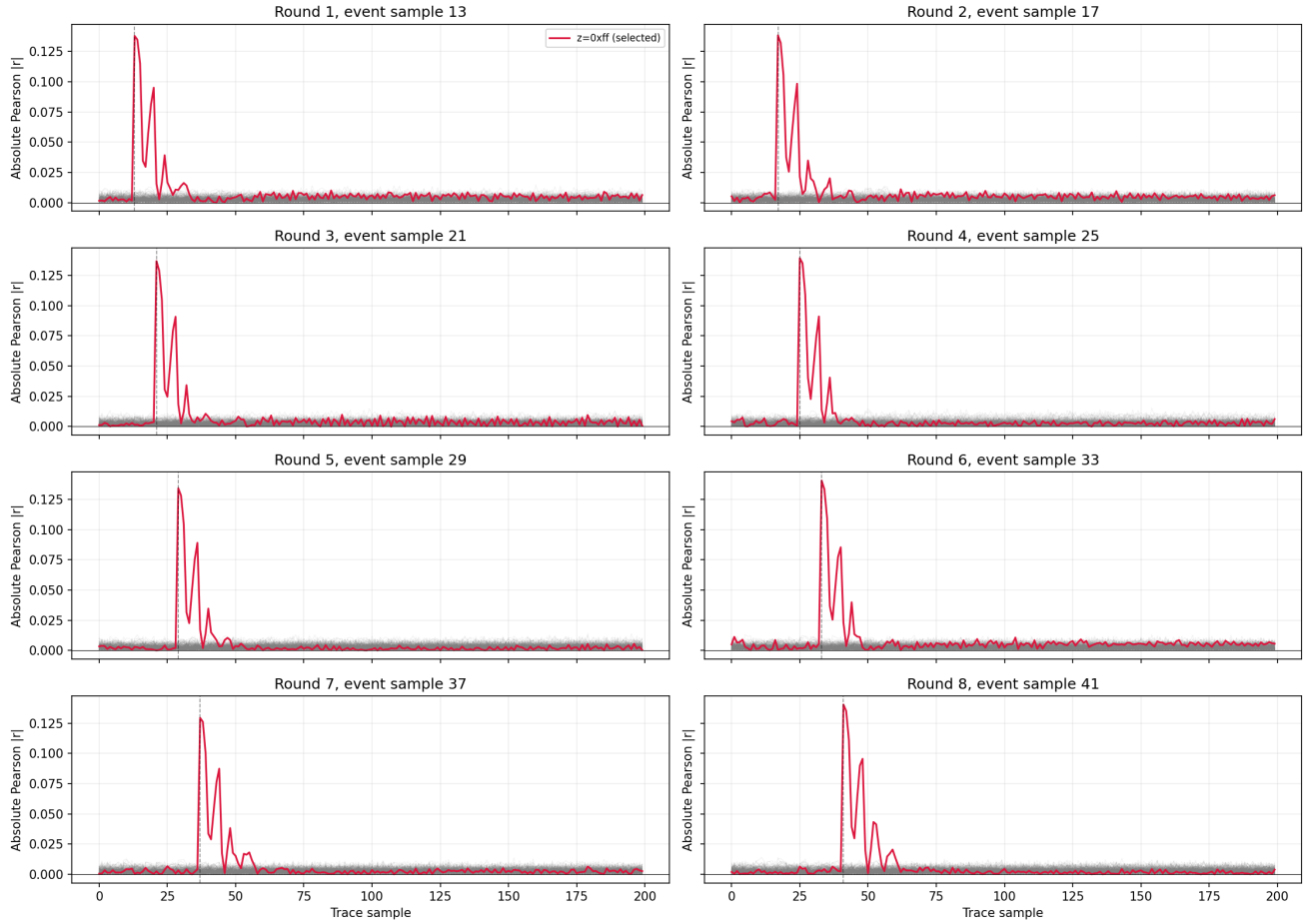


Fig. 33. Task 3: endpoint-candidate correlations for byte 13 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

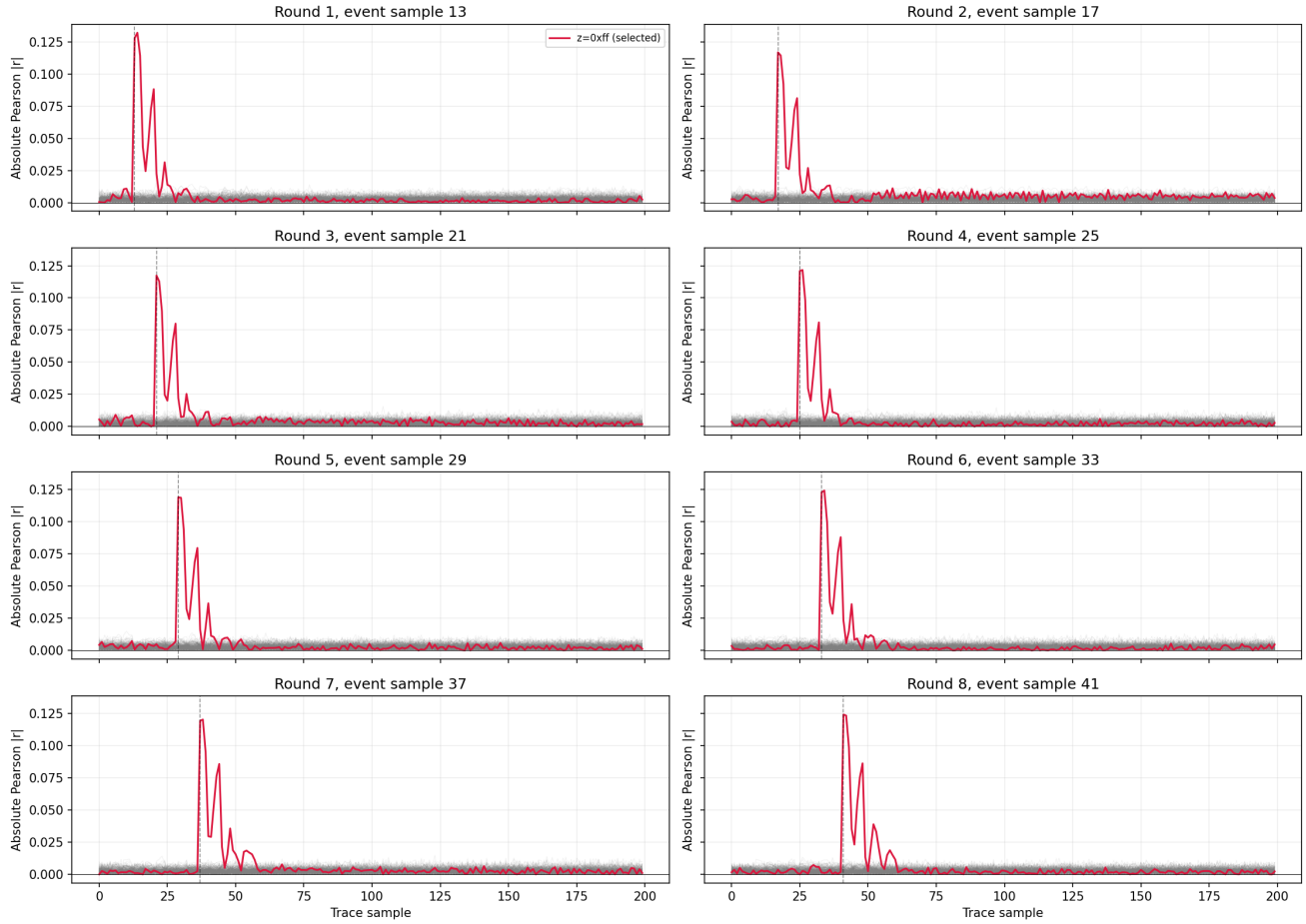


Fig. 34. Task 3: endpoint-candidate correlations for byte 14 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

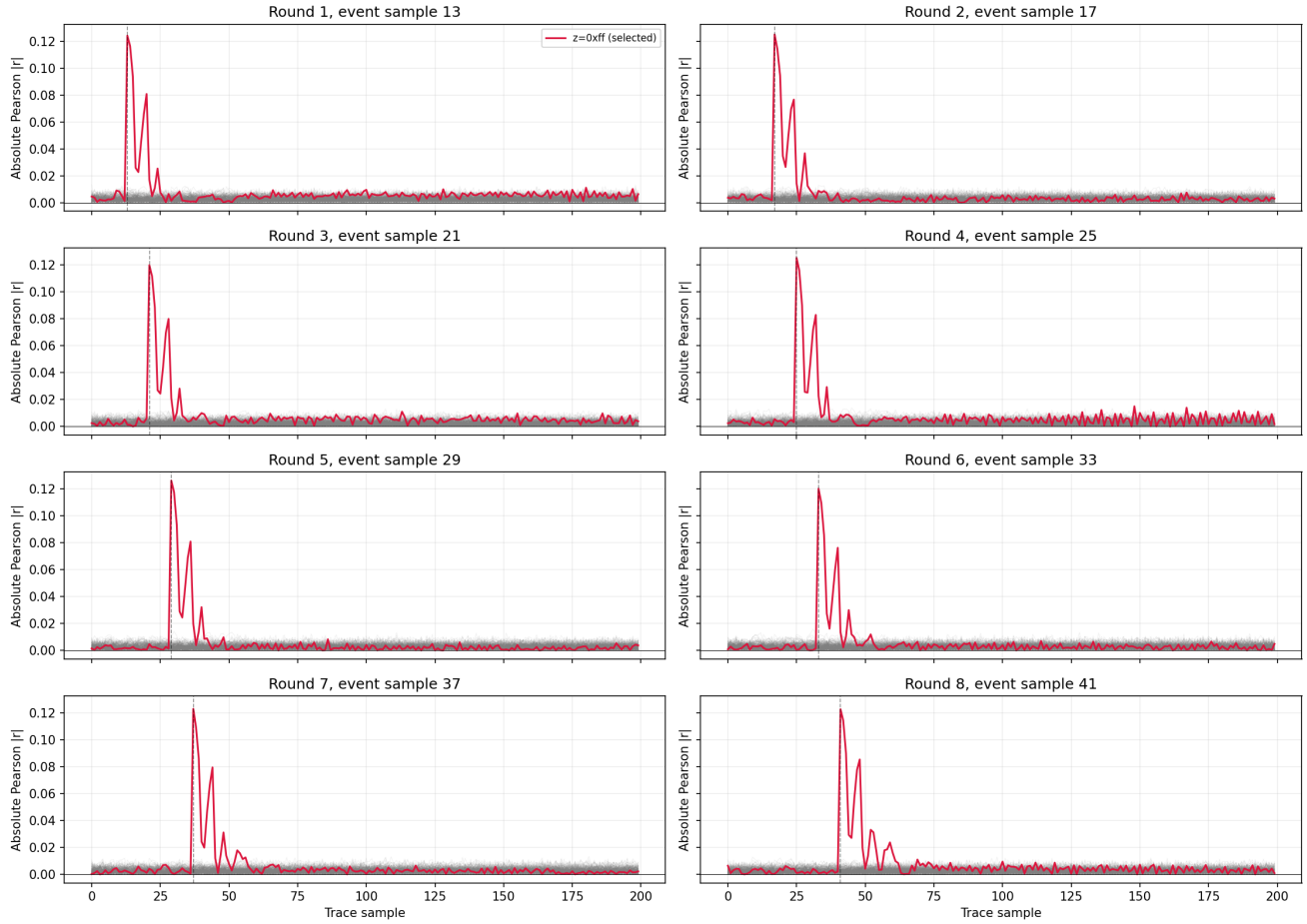


Fig. 35. Task 3: endpoint-candidate correlations for byte 15 across rounds 1–8. Red curves mark the selected $z = \texttt{0xFF}$; dashed lines mark the detected event samples.

References

[1] P. C. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Advances in Cryptology—CRYPTO '99*, Lecture Notes in Computer Science, vol. 1666. Berlin, Germany: Springer, 1999, pp. 388–397.

[2] E. Brier, C. Clavier, and F. Olivier, "Correlation power analysis with a leakage model," in *Cryptographic Hardware and Embedded Systems—CHES 2004*, Lecture Notes in Computer Science, vol. 3156. Berlin, Germany: Springer, 2004, pp. 16–29.

[3] C. O'Flynn and Z. Chen, "Side Channel Power Analysis of an AES-256 Bootloader," *Cryptology ePrint Archive*, Paper 2014/899, 2014. [Online]. Available: https://eprint.iacr.org/2014/899

[4] R. Lumbiarres-López, M. López-García, and E. Cantó-Navarro, "Faking countermeasure against side-channel attacks," in *Advances in Microelectronics: Reviews*, vol. 1, S. Y. Yurish, Ed. IFSA Publishing, 2017, ch. 19, pp. 509–526. [Online]. Available: https://upcommons.upc.edu/server/api/core/bitstreams/522a48dd-2d29-4aee-9395-aa70b82008a8/content

[5] R. Novak, "Side-channel based reverse engineering of secret algorithms," in *Proceedings of the 12th International Electrotechnical and Computer Science Conference (ERK 2003)*, 2003, pp. 445–448. [Online]. Available: https://www.researchgate.net/publication/228847690_Side-channel_based_reverse_engineering_of_secret_algorithms

[6] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197-upd1, May 2023. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.197-upd1

[7] poncho, "Answer to: In AES keyschedule, infer all round keys and cipher key from last round key," *Cryptography Stack Exchange*, Jul. 31, 2017, edited Aug. 1, 2017. [Online]. Available: https://crypto.stackexchange.com/questions/50543/in-aes-keyschedule-infer-all-round-keys-and-cipher-key-from-last-round-key